UTN FRM · INGENIERÍA EN SISTEMAS DE INFORMACIÓN · CIENCIA DE DATOS 2026

FF-15

¿Es posible predecir la duración de una partida de League of Legends a partir de las características de los jugadores, de los campeones de los equipos y las condiciones iniciales de la partida?

Documentación integral del Proyecto Integrador

Entrega 1 · Ingeniería de datos

Comisión 5K09 · Grupo 5K09-02

Exposición: miércoles 9 de septiembre, 14:55 – 15:15

`github.com/aguilarezequiel/FF-15`

Cómo usar este documento

Este documento es la memoria completa del proyecto. Está escrito para que **cualquier integrante del grupo pueda leerlo de punta a punta sin saber nada previo** y termine entendiendo qué hace el proyecto, por qué está hecho así, cómo ejecutarlo y cómo defenderlo frente al docente.

No hace falta leerlo en orden, pero conviene. Las partes I y II dan el contexto —qué pide la materia y qué teoría hay detrás—; la parte III explica la fuente de datos; la IV recorre el repositorio archivo por archivo; la V describe el dataset que se produce; la VI es la guía práctica para ejecutarlo; y la VII prepara la defensa oral.

> REGLA DE MANTENIMIENTO DE ESTE DOCUMENTO
>
> Este documento es **vivo**. Cada avance relevante del proyecto —una tarea nueva, una columna que se agrega o se quita, una decisión de diseño que cambia, un bug corregido— tiene que reflejarse acá **en el mismo movimiento**: se agrega lo nuevo y se **borra lo que dejó de ser cierto**. Una documentación que sólo acumula es peor que no tenerla, porque afirma con confianza cosas que ya no pasan.
>
> La regla está registrada en `CLAUDE.md` y en la skill del proyecto, así que cualquiera que trabaje sobre este repo con asistencia de IA la va a ver antes de tocar nada. El documento se edita en `docs/documentacion-proyecto.html` y el PDF se regenera con `python docs/generar_pdf.py`.

Estado a la fecha de esta versión

ASPECTO	ESTADO
Pipeline	Completo y funcionando. Nueve tareas, todas verificadas en verde con datos reales de la API.
Corridas	La recolección grande corrió del 5/9 11:53 al 6/9 09:33 (≈21 h 40 min). La corrida final que produjo el entregable tardó **15 minutos** de punta a punta, porque el bronce ya estaba en disco.
Bronce acumulado	**19.145** partidas crudas, **290.654** perfiles de jugador, 1.651 partidas descartadas. **1,1 GB** en disco (233 MB de partidas, 850 MB de perfiles).
Dataset (capa plata)	**19.145 filas × 137 columnas**, 11 MB. Clave única, cero nulos, ninguna columna vacía. **Supera 19 veces** el mínimo de 1.000 filas que pide la consigna.
Calidad de código	10/10 tests en verde · `ruff` sin hallazgos · `astro dev parse` limpio.
Bug corregido (2026-09-05)	`land_bronze` fallaba con `HTTP 400` en las seis plataformas: el pool de keys rotaba de key en cada request individual, y un `puuid` sólo lo puede leer la key que lo emitió. Fix: afinidad key↔puuid en `RiotClient` — ver 4.1.
Incidente superado (2026-09-06)	La máquina entró en hibernación a las 03:19 con cuatro tareas `land_bronze` a mitad de camino. Los procesos murieron; **el bronce no perdió un solo archivo**. Airflow relanzó las tareas solo a las 07:16 y el dataset se generó igual. Es la mejor evidencia práctica de por qué el bronce va a disco y es append-only — ver 7.7.
Limitación conocida	**Sesgo de muestreo por tier.** Cada plataforma quedó con partidas de un rango casi puro y distinto (elo medio de 1.627 en NA1 a 3.231 en LA1). Está medido y declarado en 7.5; no invalida el dataset, pero condiciona qué conclusiones se pueden sacar de él.
Lo que falta	Nada bloqueante para la Entrega 1: el dataset está generado y verificado. Queda ensayar la defensa con el reparto de la parte VII.

Contenido

Parte I · El proyecto dentro de la materia

- Qué es el Proyecto Integrador y cuánto pesa
- Las cuatro entregas y el calendario
- La pregunta de investigación
- Los siete criterios de una buena fuente, aplicados a nuestro caso
- Qué pide exactamente la Entrega 1

Parte II · La teoría de la Unidad 1, explicada desde cero

- Qué es la ingeniería de datos y dónde encaja
- Pipelines, ETL y ELT
- Qué problema resuelve un orquestador
- Apache Airflow: DAG, tarea, operador, hook, provider
- Las piezas de Airflow 3 y qué hace cada una
- Astro Runtime y Astro CLI: qué es cada cosa

El modelo medallón: bronce, plata y oro
Idempotencia, y por qué importa tanto acá
Datos tidy

Parte III · La fuente de datos: la API de Riot Games

Qué publica Riot y cómo se accede
La API key: tipos, límites y vencimiento
Las dos familias de hosts, el detalle que más confunde
Los siete endpoints que usa el pipeline
Data Dragon: el catálogo público sin key
El recorrido de semilla a dataset

Parte IV · El repositorio, archivo por archivo

Árbol completo y criterio de organización
Configuración del entorno
Credenciales y seguridad
include/lol/ — la lógica del dominio
dags/lol_ingest.py — la orquestación
Tests, calidad de código y documentación

Parte V · El dataset producido

Unidad de análisis, clave y objetivo
Las 137 columnas por bloques
Reglas de admisión de una partida
Variables derivadas y su justificación
Los nulos: dónde, por qué y qué se hizo
Las capas en disco

Parte VI · Guía de ejecución

Requisitos previos
Puesta en marcha desde cero
Obtener y renovar la API key
Cómo llegar a 20.000 filas
Qué mirar mientras corre
Problemas frecuentes y cómo resolverlos

Parte VII · La defensa de la Entrega 1

Cómo se estructuran los 20 minutos
Las seis verificaciones numéricas
Las cinco preguntas que se explican
La confesión sobre tidy
Limitaciones que declaramos

Reparto de conocimiento en el grupo

Banco de preguntas probables

Anexo · Glosario

PARTE I

El proyecto dentro de la materia

1. Qué es el Proyecto Integrador y cuánto pesa

El Proyecto Integrador es un proyecto de ciencia de datos *de punta a punta*, sobre un problema que elige el grupo y datos que consigue el grupo. Empieza el 12 de agosto y termina el 18 de noviembre. En ese recorrido hay que conseguir los datos y automatizar su descarga, explorarlos, modelarlos, visualizarlos y comunicar los hallazgos en una aplicación funcionando.

No es un anexo de la materia: es el eje. Tres números que lo dimensionan, tomados del *Kit de arranque*:

- Vale el **50 % de la nota final** — el doble que los cuatro trabajos prácticos juntos, y el doble que los dos parciales.
- Ocupa **cuatro clases completas** del cuatrimestre, dedicadas exclusivamente a las entregas.
- Cada TP práctico está diseñado para preparar la entrega del integrador que viene después.

Se aprueba con el 60 %. Hay una instancia de recuperación al final del cursado.

> **LOS TP Y EL INTEGRADOR NO SON LO MISMO**
>
> Los cuatro TP prácticos se hacen sobre el dataset **FIFA**, que provee la cátedra: es un caso controlado donde se aprende la técnica. El Integrador aplica esa misma técnica al dataset propio, que va a ser más sucio, más ambiguo y más difícil. Por eso conviene hacer el TP *antes* de encarar la entrega correspondiente del integrador. El orden importa.

2. Las cuatro entregas y el calendario

ENTREGA	FECHA	QUÉ SE PRESENTA
Definición	12/08	Grupo, pregunta y fuente de datos
1 · Ingeniería de datos	**miércoles 9/9**	Pipeline automatizado que produce tu dataset
2 · Análisis exploratorio	16/09	Exploración, hipótesis y hallazgos
3 · Modelado	14/10	Objetivo predictivo, modelos comparados y métricas
4 · Visualización e integración	04/11	Visualizaciones y aplicación funcionando
Exposición final	18/11	Proyecto completo y demo en vivo
Recuperación	25/11	Para los grupos que la necesiten

> **OJO CON LA FECHA DE LA ENTREGA 1**
>
> El PDF del *Kit de arranque* dice que la Entrega 1 es el **02/09**. La página del campus, que es más reciente, dice **miércoles 9 de septiembre** y publica las franjas horarias por grupo. **Vale la del campus.** Al grupo 5K09-02 le toca de **14:55 a 15:15**. Hay que llegar cinco minutos antes con el entorno ya levantado: la franja empieza y termina a la hora que dice, y lo que no se llega a mostrar no se recupera.

Cada entrega parcial es una reunión privada con el docente, sin audiencia de otros grupos. La exposición final del 18/11 sí es pública, frente al curso.

EL ERROR DE GESTIÓN MÁS COMÚN

Trabajar el integrador la semana previa a cada entrega. Las entregas están separadas por tres o cuatro semanas y esa distancia genera la ilusión de que sobra tiempo. No sobra: cada entrega se apoya en la anterior, y llegar con lo mínimo a la Entrega 1 significa arrastrar un pipeline frágil hasta noviembre.

3. La pregunta de investigación

> NUESTRA PREGUNTA
>
> **¿Es posible predecir la duración de una partida de League of Legends a partir de las características de los jugadores, de los campeones de los equipos y las condiciones iniciales de la partida?**

La materia insiste en un orden que no es obvio: **primero la pregunta, después la fuente**. Elegir un dataset primero e inventar la pregunta después lleva a proyectos mecánicos que se hacen cuesta arriba a partir de septiembre.

Vale la pena desarmar nuestra pregunta, porque cada parte tiene consecuencias técnicas concretas:

PARTE DE LA PREGUNTA	CONSECUENCIA EN EL PROYECTO
«la duración de una partida»	Define la **unidad de análisis**: una fila es una partida. Y define la **variable objetivo**: `game_duration_s`, en segundos.
«predecir»	Como el objetivo es un número continuo, es un problema de **regresión** (no de clasificación). Eso ya define qué modelos se comparan en la Unidad 3.
«características de los jugadores»	Hay que traer datos de los 10 jugadores: su rango, su winrate, su experiencia con el campeón que eligieron.
«campeones de los equipos»	Hay que traer el catálogo de campeones y derivar métricas de composición de equipo.
«condiciones iniciales»	Es la restricción más importante: **sólo entra lo que se conoce antes de que la partida empiece**. Nada de kills, oro, torres ni ningún evento del juego.

> POR QUÉ LA RESTRICCIÓN DE «CONDICIONES INICIALES» NO ES NEGOCIABLE
>
> Si el dataset incluyera, por ejemplo, la cantidad de torres destruidas, el modelo predeciría la duración con una precisión altísima… y sería inútil, porque para saber cuántas torres cayeron hay que esperar a que la partida termine. Predecir el pasado no sirve. En la jerga de ciencia de datos esto se llama **fuga de información** (*data leakage*), y es uno de los errores más caros y más difíciles de detectar, porque el modelo «funciona» espectacularmente bien en las pruebas y falla por completo en la realidad.

4. Los siete criterios de una buena fuente, aplicados a nuestro caso

En la primera clase la cátedra dio siete criterios para *elegir* una fuente de datos. Para la Entrega 1 esos mismos siete criterios se usan para *juzgar lo que produjo el pipeline*. Cambió una sola cosa, y no es menor: en agosto la calidad del dato era responsabilidad de quien publicó la fuente; ahora es nuestra.

Los cuatro primeros son **eliminatorios** — si la fuente falla uno solo, no hay proyecto. Los tres últimos son **facilitadores**: no lo rompen, pero lo complican, y cada uno tiene una salida de emergencia si se pone más trabajo.

LOS CUATRO ELIMINATORIOS

#	CRITERIO	QUÉ SIGNIFICA	NUESTRO CASO
1	**Datos tidy**	Cada fila es una observación, cada columna una variable.	**Cumple, con una concesión declarada.** Una fila es una partida, que es la unidad de la pregunta. Pero los 10 jugadores están aplanados en 100 columnas en vez de estar en formato largo. Está explicado y justificado en la parte VII.4.
2	**Unidad alineada con la pregunta**	La unidad de cada fila coincide con aquello sobre lo que se quiere concluir.	**Cumple.** La pregunta es sobre la duración de *una partida*, y una fila es exactamente una partida, identificada por `match_id`.
3	**Algo modelable**	Hay una variable a predecir, o grupos naturales que se sospecha encontrar.	**Cumple por partida doble.** Supervisado: regresión sobre `game_duration_s`. No supervisado: arquetipos de composición de equipo a partir de las columnas de agregados.
4	**Descargable de forma automatizada**	URL, API o página scrapeable. Sin intervención humana.	**Cumple.** API REST oficial de Riot Games, con key, orquestada en Airflow. Cero pasos manuales dentro del pipeline.

LOS TRES FACILITADORES

#	CRITERIO	OBJETIVO	NUESTRO CASO
5	**Volumen suficiente**	Más de 1.000 filas; ideal más de 10.000.	**Alcanzable.** El objetivo del pipeline es parametrizable. Con 20.000 partidas el dataset queda en zona «cómodo» según la escala de la cátedra. Es lo único que falta ejecutar.
6	**Varias columnas informativas**	Cinco o más útiles, mezclando tipos.	**Cumple holgado.** 137 columnas: 106 enteras, 20 flotantes y 11 de texto categórico.
7	**Documentación entendible**	Saber qué significa cada columna y sus unidades.	**Cumple.** Documentación oficial del Riot Developer Portal, más el catálogo Data Dragon. La parte V de este documento traduce cada columna.

La escala de volumen que usa la cátedra: menos de 1.000 filas es *insuficiente*; entre 1.000 y 10.000 es *suficiente para arrancar*; más de 10.000 es *cómodo*.

ERRORES DE AÑOS ANTERIORES QUE ESTE PROYECTO EVITA

ERROR TÍPICO	POR QUÉ NO NOS PASA
Datos ya agregados (promedios por provincia, totales por año)	Trabajamos con la observación individual: una partida concreta, con sus 10 jugadores identificados.
Sin variable objetivo ni estructura latente	`game_duration_s` es un objetivo explícito y continuo.
Muy pocas filas	La fuente es prácticamente inagotable: se juegan millones de partidas por día.
Una sola columna útil	137 columnas, de cuatro naturalezas distintas (jugador, campeón, composición, contexto).
Bajable sólo a mano	API REST pública y documentada.
Fuente elegida por descarte, sin interés real	Es un tema que el grupo eligió y conoce, lo cual pesa: son cuatro meses de convivencia con esta decisión.

5. Qué pide exactamente la Entrega 1

La consigna del campus lo resume en una sola pregunta de fondo:

> **«¿Tenés un pipeline que produce tu dataset sin que nadie lo toque a mano?»** Todo lo demás son formas de responderla.

LO QUE HAY QUE LLEVAR EL DÍA DE LA ENTREGA

QUÉ	DETALLE
El entorno levantado	Airflow corriendo en la notebook de alguien del grupo, con la interfaz abierta en el navegador. Prendido *antes* de entrar.
Una corrida en verde	Una ejecución completa y exitosa, **ya hecha**, visible en el historial. No se corre en el momento.
El CSV producido	El archivo que generó esa corrida, abierto o listo para abrir.
El chequeo de calidad	Los criterios verificados sobre la salida, con los números a mano.

CÓMO SE REPARTEN LOS 20 MINUTOS

MINUTOS	MOMENTO	QUÉ PASA
0–2	De qué se trata	La pregunta del proyecto en una frase, la fuente, y qué es una fila. Tres oraciones, sin filminas.
2–7	La corrida	Se muestra la ejecución en verde y se recorre el grafo explicando qué hace cada tarea y por qué está donde está.
7–12	El dataset	Se abre el CSV y se lo defiende contra los siete criterios: cuántas filas, cuál es la clave, dónde hay nulos, cuál es la columna objetivo.
12–18	Preguntas dirigidas	El docente pregunta **a integrantes puntuales** sobre cualquier parte de lo anterior.
18–20	Devolución	Qué está bien y qué corregir antes de la Entrega 2.

> **LA EVALUACIÓN ES INDIVIDUAL**
>
> Repartirse el trabajo está bien; desentenderse de la parte del otro, no. La pregunta sobre la extracción puede caerle a quien escribió la transformación. Con grupos de cuatro a siete personas esto pesa: **no alcanza con que uno solo sepa todo**. Por eso la parte VII de este documento incluye un reparto sugerido y un banco de preguntas.

LOS CINCO PUNTOS CON LOS QUE SE APRUEBA

1. Hay una corrida completa en verde, hecha por ustedes, visible en la interfaz.
2. El grupo sabe explicar qué hace cada tarea del pipeline y por qué está ahí.
3. El pipeline produce un CSV que cumple los criterios medibles.
4. El grupo sabe defender las decisiones de diseño.
5. Las respuestas no vienen de una sola persona.

LO QUE LA CONSIGNA DICE EXPLÍCITAMENTE QUE NO HAY QUE HACER

- **Llegar sin el entorno levantado.** Arrancar Airflow delante del docente se lleva la mitad del tiempo, y es tiempo que no se recupera.
- **Correr el pipeline en vivo.** Hay que llevar la corrida hecha. Si se quiere mostrar una ejecución en el momento, que sea sobre una muestra chica y *además* de la que ya está.
- **Que hable uno solo.** Es el error más caro, porque las preguntas van dirigidas y la nota es individual.

Y una nota final de la consigna que conviene tener presente: si algo del pipeline no llegó a funcionar, hay que ir igual y decirlo. Un grupo que muestra un pipeline a medias y entiende dónde está trabado se lleva una devolución útil. Uno que no se presenta, no.

PARTE II

La teoría de la Unidad 1, explicada desde cero

Esta parte no da nada por sabido. Si ya conocés Airflow, se puede saltear hasta la parte III. Si no, acá está todo lo que hace falta para entender por qué el proyecto está construido como está.

1. Qué es la ingeniería de datos y dónde encaja

La materia arranca con una idea que suena obvia pero cambia todo: **la ciencia de datos no empieza con el análisis, sino mucho antes, con los datos mismos**. Antes de que alguien pueda hacer un gráfico o entrenar un modelo, alguien tuvo que conseguir los datos, limpiarlos, guardarlos en algún lado y dejarlos disponibles de forma confiable y repetible.

Ese «alguien» es el rol de **ingeniería de datos**, y su producto no es un informe: es un *sistema* que produce datos. La diferencia importa. Un análisis se hace una vez; un sistema tiene que funcionar mañana, y pasado, y cuando la fuente cambie de formato.

La bibliografía de la unidad es *Fundamentals of Data Engineering* (Reis y Housley), que organiza el trabajo alrededor del **ciclo de vida del dato**: generación, almacenamiento, ingesta, transformación y disponibilización. Nuestro proyecto cubre las primeras cuatro; la quinta —que el dato esté disponible para quien lo consuma— es lo que van a usar las Unidades 2, 3 y 4.

2. Pipelines, ETL y ELT

Un **pipeline** (tubería) es una secuencia de pasos por la que pasa el dato desde su origen hasta su destino. Cada paso toma algo, lo transforma y se lo pasa al siguiente.

Las dos formas clásicas de armar un pipeline se distinguen por el orden de las letras:

ENFOQUE	ORDEN	IDEA
ETL	Extract → Transform → Load	Se extrae el dato, se lo transforma *en el camino* y recién después se lo guarda. Se guarda sólo el resultado final.
ELT	Extract → Load → Transform	Se extrae el dato y se lo guarda **crudo** primero. La transformación se hace después, sobre lo que ya está guardado.

> NUESTRO PROYECTO ES ELT, Y ES UNA DECISIÓN CON CONSECUENCIAS
>
> Guardamos primero la respuesta cruda de la API (capa bronce) y recién después la transformamos en filas (capa plata). La ventaja concreta: si mañana descubrimos que veníamos interpretando mal una columna, corregimos el código de transformación y **reprocesamos lo que ya está en disco**, sin volver a pedirle nada a Riot. Con ETL habría que volver a descargar todo — y con una API que permite 100 pedidos cada 2 minutos, eso significa horas perdidas.

3. Qué problema resuelve un orquestador

Imaginemos el pipeline como un script de Python que hace todo de corrido. Funciona… hasta que:

- Falla el paso 7 de 9, y hay que volver a correr los 6 anteriores que ya habían salido bien.

- Hay que correrlo todos los días a las 8, y alguien se tiene que acordar.
- Un paso tarda tres horas y nadie sabe si está avanzando o colgado.
- Dos pasos son independientes y podrían correr en paralelo, pero el script los hace en fila.
- Falló hace tres días y nadie se enteró.

Un **orquestador** resuelve exactamente esa lista. No hace el trabajo: coordina quién lo hace, cuándo, en qué orden, qué pasa si falla y cómo se ve el estado. Apache Airflow es el orquestador que usa la industria y el que enseña la materia.

> UN EJEMPLO CONCRETO DE NUESTRO PROYECTO
>
> El 6/9 la máquina entró en hibernación a las 03:19, en medio de una corrida de 14 horas, con cuatro tareas de descarga a mitad de camino. Los procesos murieron en seco. **Las 16.908 partidas ya descargadas quedaron intactas en disco**, el scheduler detectó las tareas huérfanas y las relanzó solo a las 07:16, y el dataset se generó ese mismo día. Con un script monolítico habría que haber empezado de cero. Eso es lo que compra un orquestador bien usado.

4. Apache Airflow: DAG, tarea, operador, hook, provider

DAG

Es la sigla de *Directed Acyclic Graph*: grafo dirigido acíclico. Suena peor de lo que es. Un DAG es simplemente **el dibujo del flujo de trabajo**: cajitas (las tareas) unidas por flechas (las dependencias).

- **Dirigido** porque las flechas tienen sentido: A va antes que B.
- **Acíclico** porque no puede haber ciclos: si A depende de B y B depende de A, ninguna de las dos podría empezar nunca.

En Airflow, un DAG se escribe en Python. Nuestro DAG se llama `lol_ingest` y vive en `dags/lol_ingest.py`.

TAREA (TASK)

Cada cajita del grafo. Es una unidad de trabajo que se ejecuta, tiene su propio log, su propio estado (éxito, falla, salteada) y sus propios reintentos. Nuestro DAG tiene nueve.

OPERADOR

Una plantilla de tarea que ya viene resuelta. Por ejemplo, `HttpOperator` hace un pedido HTTP: uno lo configura en vez de escribirlo. La cátedra da una regla para elegir entre las tres formas:

FORMA	CUÁNDO CONVIENE
Operador prefabricado	La acción es estándar y alguien ya la resolvió bien.
`@task` + hook	Se necesita lógica propia, pero se quiere reusar la conexión y el manejo de errores.
`@task` pelado	No hay sistema externo: transformar, validar, guardar.

Nuestro proyecto usa **casi todo `@task` pelado**, porque el cliente de Riot necesita una lógica de rotación de claves y control de velocidad que ningún operador prefabricado resuelve. Esa decisión hay que poder

justificarla, y la justificación es exactamente ésa.

HOOK

Es la capa de abajo de un operador: el pedacito que sabe hablar con el sistema externo y manejar la conexión. Se usa cuando se quiere la conexión pero no el operador entero.

PROVIDER

Un paquete que agrega integraciones con sistemas externos. De fábrica Airflow trae sólo `apache-airflow-providers-standard` y `common-sql`. El resto se agrega a `requirements.txt`. La cátedra deja una recomendación: *antes de escribir código para hablar con tu fuente, fijate si hay un provider que ya lo resuelva*.

TASKFLOW API

Es la forma moderna de escribir DAGs: se usan **decoradores** (`@dag`, `@task`) y las dependencias se deducen solas del flujo de datos entre funciones. La forma antigua —instanciar operadores y unirlos con `>>`— sigue siendo válida y es lo que aparece en la mayoría de los tutoriales que se googlean, porque hay mucho material escrito para Airflow 2. Conviene saber leerla, pero para escribir código nuevo se usa la de decoradores.

XCOM

Es el mecanismo por el que dos tareas se pasan datos. Tiene una regla de oro que la cátedra subraya: **por XCom viajan metadatos, no datos** — rutas, identificadores, banderas, conteos. Los datos van a disco.

> POR QUÉ ESA REGLA EXISTE
>
> XCom guarda lo que se le pasa en la base de datos de metadatos de Airflow. Si se le mandan miles de filas, esa base se satura y el orquestador entero se degrada. Es uno de los errores más comunes al empezar. En nuestro pipeline, la tarea que descarga devuelve una *lista de identificadores de partidas* y las partidas mismas quedan en archivos.

SENSORES, RAMIFICACIÓN Y CORTOCIRCUITO

HERRAMIENTA	PARA QUÉ SIRVE	DÓNDE LA USAMOS
Sensor (`@task.sensor`)	Esperar algo que está *fuera* del control del DAG: que un sitio vuelva, que aparezca un archivo.	`wait_for_riot` espera a que la API responda.
Rama (`@task.branch`)	Elegir el camino en tiempo de ejecución devolviendo el nombre de la tarea siguiente.	No la usamos: nuestro flujo no se bifurca en dos caminos alternativos.
Cortocircuito (`@task.short_circuit`)	Si devuelve `False`, todo lo que está aguas abajo queda *salteado* y la corrida termina **bien**.	`check_key` corta la corrida si la API key no sirve.

Dos conceptos asociados que aparecen en nuestro código:

- `mode="reschedule"` en un sensor: en vez de ocupar un trabajador durmiendo entre sondeos, la tarea se libera y Airflow la vuelve a encolar más tarde. Con un sensor da igual; con cincuenta esperando, es

la diferencia entre un orquestador sano y uno tapado.

- **Trigger rules**: por defecto una tarea corre si todas sus predecesoras salieron bien (`ALL_SUCCESS`). Nuestro `check_key` usa `ALL_DONE`, que significa «corré igual, hayan salido bien o mal». Es necesario porque su tarea anterior puede quedar *salteada*, y ése es justamente el caso que tiene que manejar: con la regla por defecto no se ejecutaría nunca cuando más falta hace.

MAPEO DINÁMICO DE TAREAS

A veces no se sabe de antemano cuántas tareas hacen falta. `.expand()` crea una instancia de la tarea por cada elemento de una lista, y todas corren en paralelo. Nosotros lo usamos para generar **una tarea por plataforma de juego**: en la interfaz se ven seis instancias de `land_bronze`, una por servidor, cada una con su propio log y su propio estado.

5. Las piezas de Airflow 3 y qué hace cada una

Airflow no es un programa: son varios procesos que se coordinan a través de una base de datos. Ninguno le habla directo a otro.

PROCESO	QUÉ HACE
`dag-processor`	Lee los archivos `.py` de `dags/` cada pocos segundos y registra los DAGs que encuentra. **En Airflow 2 vivía dentro del scheduler; en la 3 es un proceso aparte.**
`scheduler`	Decide qué tarea está lista para correr y la manda a ejecutar.
`api-server`	La interfaz web y la API REST. **En Airflow 2 se llamaba `webserver`.**
`triggerer`	Maneja las esperas asincrónicas (los sensores en modo diferido).
base de metadatos	Guarda el estado de todo. En nuestro entorno local es un PostgreSQL que levanta solo.

> **POR QUÉ ESTO IMPORTA PARA ESCRIBIR CÓDIGO**
>
> El `dag-processor` parsea la carpeta `dags/` **cada pocos segundos**. Todo lo que sea lento de importar hace más lento el parseo de todo el proyecto. De ahí sale una regla que aplicamos en todo el código: `import pandas` va *adentro* de la función de la tarea, nunca arriba del archivo, y toda la lógica pesada vive en `include/` y se importa.

AIRFLOW 2 CONTRA AIRFLOW 3

La materia usa **Airflow 3.3**. Casi todo lo que se encuentra googleando está escrito para Airflow 2 y no compila. Ésta es la tabla de traducción que hay que tener a mano:

AIRFLOW 2	AIRFLOW 3
`from airflow.decorators import dag, task`	`from airflow.sdk import dag, task`
`from airflow.utils.trigger_rule import TriggerRule`	`from airflow.task.trigger_rule import TriggerRule`
`schedule_interval=...`	`schedule=...`
`context["execution_date"]`	`context["logical_date"]`
`Variable.get(k, default_var=None)`	`Variable.get(k, default=None)`
servicio `webserver`	servicio `api-server`
el scheduler parsea los DAGs	lo hace el `dag-processor`, aparte

6. Astro Runtime y Astro CLI: qué es cada cosa

Esto confunde mucho al principio, así que conviene separarlo con precisión:

COSA	QUÉ ES	¿SE PUEDE VIVIR SIN ELLA?
Apache Airflow	El orquestador. El software en sí.	No. Es el corazón.
Providers	Integraciones con sistemas externos.	Sí, pero casi siempre se quiere alguno.
Astro Runtime	Una *imagen de Docker* con Airflow ya armado y probado.	Sí: se puede instalar Airflow a mano.
Astro CLI	El comando que levanta esa imagen en tu máquina.	Sí: se puede usar Docker directo.
Plugins	Extensiones de la interfaz.	Sí. En esta materia no se escribe ninguno.

Dicho corto: `astro dev start` **no es un comando de Airflow, es de Astro**. Se usa Astro porque ahorra una tarde de configuración: sin él habría que armar a mano la base de datos y los cuatro procesos.

7. El modelo medallón: bronce, plata y oro

Es una forma de organizar los datos en capas, cada una más elaborada que la anterior. El nombre viene de las medallas y el orden es el de valor creciente en refinamiento.

CAPA	QUÉ CONTIENE	REGLA	EN NUESTRO PROYECTO
Bronce	El dato *tal como llegó* de la fuente. Sin interpretar, sin limpiar, sin tocar.	Sólo se agrega, nunca se modifica ni se reescribe.	Las respuestas JSON de la API de Riot, comprimidas, en `include/output/bronze/`.
Plata	Datos limpios, tipados, una fila por observación, validados.	Se reconstruye siempre desde el bronce.	El CSV de 137 columnas. **Es el entregable de la Entrega 1.**
Oro	Datos preparados para un uso específico: variables derivadas, agregados, tablas para un tablero.	Se construye sobre la plata.	**No se construye acá.** Es lo que se arma en las Unidades 3 y 4.

> POR QUÉ EL BRONCE ES LA PARTE QUE MÁS SE SUBESTIMA
>
> La consigna de la Entrega 1 pregunta literalmente: *«¿Dónde guardás el dato crudo? Si mañana descubrís que venías interpretando mal una columna, ¿podés arreglarlo sin volver a la fuente? El dataset que entregás es la capa plata; la pregunta es si además tenés el bronce.»*
>
> Nuestra respuesta es sí, y es más completa de lo mínimo: el bronce no guarda sólo las partidas, guarda **todas** las respuestas crudas —partidas, entradas de liga, maestrías y el catálogo de campeones—. Gracias a eso la capa plata se reconstruye **100 % sin red**.

8. Idempotencia, y por qué importa tanto acá

Una operación es **idempotente** si ejecutarla muchas veces da el mismo resultado que ejecutarla una vez. Apagar una luz es idempotente: si ya está apagada, apagarla de nuevo no cambia nada. Sumar uno no lo es.

En un pipeline esto se traduce en una pregunta muy concreta que la consigna hace textualmente: *«¿Qué pasa si lo corrés de nuevo? ¿Sale el mismo archivo? Si no, ¿por qué?»*

> NUESTRA RESPUESTA, Y HAY QUE DARLA COMPLETA
>
> Hay que separar dos cosas que se confunden fácil:
>
> - **De la transformación para abajo, el pipeline es idempotente.** Con el mismo bronce en disco, produce siempre exactamente el mismo CSV. Es una función pura del disco: no hay azar, no hay red, no hay reloj.
> - **La descarga es acumulativa, a propósito.** Cada corrida sale a buscar partidas que todavía no tiene, hasta llenar su cuota. Las que ya están nunca se vuelven a pedir; las nuevas se suman.
>
> O sea: **no sale el mismo archivo, sale uno más grande**. Y es deliberado: es la única forma de juntar volumen con una API que permite 100 pedidos cada 2 minutos por región. Si se quisiera congelar el dataset, alcanza con no volver a correr la descarga y reprocesar el bronce tal como está.

9. Datos tidy

Tidy significa **ordenado**, y define una forma canónica de organizar una tabla:

- Cada fila es **una observación**.

- Cada columna es **una variable**.

El ejemplo clásico de la cátedra:

FORMATO ANCHO (NO TIDY)

```
Persona | Enero | Febrero | Marzo
Ana     |  1500 |    1700 |  1600
Bruno   |  1300 |    1400 |  1450
```

«Mes» es una variable, pero está escondida como nombre de columna.

FORMATO LARGO (TIDY)

```
Persona | Mes     | Valor
Ana     | Enero   | 1500
Ana     | Febrero | 1700
Ana     | Marzo   | 1600
Bruno   | Enero   | 1300
```

Cada fila es una observación persona-mes.

La cátedra hace una confesión que vale la pena copiar en espíritu: **su propio dataset canónico no cumple del todo la regla**. La columna `player_positions` del dataset FIFA guarda más de un valor en la misma celda en el 65 % de las filas. No es un descuido: es una concesión deliberada, porque separarlo complicaría todo lo demás.

Y deja la instrucción explícita: *si tu dataset tiene una concesión parecida, no la escondas: explicala. Un grupo que sabe dónde su dataset se aparta del ideal y por qué está mejor parado que uno que repite la definición de memoria*. Nuestra concesión está desarrollada en la parte VII.4.

PARTE III

La fuente de datos: la API de Riot Games

1. Qué publica Riot y cómo se accede

Riot Games —la empresa que hace League of Legends— publica una API REST oficial en `developer.riotgames.com`. A través de ella se puede consultar, entre otras cosas:

- El **ranking** de jugadores de cada servidor, por rango.
- La **lista de partidas** recientes de un jugador.
- El **detalle completo** de una partida: quiénes jugaron, con qué campeones, cuánto duró.
- La **maestría** de un jugador con cada campeón (cuánto lo jugó históricamente).

Es una API REST: se le hace un pedido HTTP a una dirección y devuelve JSON. La autenticación es por un encabezado `X-Riot-Token` con la API key.

> POR QUÉ ESTA FUENTE CUMPLE EL CRITERIO 4 DE LA CÁTEDRA
>
> El criterio eliminatorio «descargable de forma automatizada» acepta explícitamente *«APIs REST públicas, con o sin API key, siempre que se pueda automatizar»*. Es exactamente nuestro caso, y con una ventaja sobre el scraping: la fuente es estable, está documentada y no se rompe cuando alguien cambia el HTML de una página.

2. La API key: tipos, límites y vencimiento

TIPO DE KEY	LÍMITES	VENCIMIENTO
Development (la que usamos)	20 pedidos por segundo · 100 pedidos cada 2 minutos	**Cada 24 horas.** Hay que regenerarla en el portal.
Personal	Los mismos límites	No vence, pero requiere aprobación de Riot.
Production	500 cada 10 s · 30.000 cada 10 min	No vence. Requiere un proyecto aprobado.

> LAS DOS CARACTERÍSTICAS QUE CONDICIONAN TODO EL DISEÑO
>
> **Vence cada 24 horas.** Por eso el DAG se dispara a mano y no está programado: un DAG programado corriendo sin nadie mirando fallaría todos los días apenas venciera la key, y el ruido de esas fallas taparía las reales. Y por eso lo primero que hace el pipeline es verificar que la key sirve — enterarse a la hora seis de una corrida de ocho es el peor resultado posible.
>
> **Los límites se aplican por región, no globalmente.** Ésta es la propiedad que hace viable el proyecto, y está explicada abajo.

EL POOL DE KEYS

Como el límite es por key *y* por región, tener varias keys multiplica el cupo. Nuestro cliente acepta **una lista de keys separadas por coma** —una por integrante del grupo— y reparte los pedidos entre ellas. Con cuatro keys la recolección va aproximadamente cuatro veces más rápido.

3. Las dos familias de hosts, el detalle que más confunde

Riot tiene **dos tipos de direcciones** y usar la equivocada devuelve un error 404 que parece un bug del código. Es probablemente el detalle técnico más importante de toda esta parte.

FAMILIA	EJEMPLOS DE HOST	QUÉ ENDPOINTS VIVEN AHÍ
Plataforma (un servidor de juego)	`la2`, `la1`, `br1`, `na1`, `euw1`, `kr`	Ranking (league-v4), perfiles, maestrías (champion-mastery-v4)
Región continental	`americas`, `europe`, `asia`	**Todo lo de partidas** (match-v5)

El mapeo de plataforma a región que usa el proyecto:

PLATAFORMA	REGIÓN	NOMBRE
`la2`	`americas`	Latinoamérica Sur
`la1`	`americas`	Latinoamérica Norte
`br1`	`americas`	Brasil
`na1`	`americas`	Norteamérica
`euw1`	`europe`	Europa Oeste
`kr`	`asia`	Corea

> LA CONSECUENCIA PRÁCTICA, MEDIDA EN NUESTRO PROYECTO
>
> Como cada host tiene su propio cupo, correr las **seis plataformas en paralelo** multiplica por seis la velocidad en los endpoints de perfil y maestría — que son justamente el cuello de botella, porque hay que consultar 10 jugadores por partida.
>
> Pero atención a la sutileza: `la2`, `la1`, `br1` y `na1` **comparten** el cupo de partidas, porque las cuatro enrutan a `americas`. En la práctica no molesta, porque las partidas son ~1 pedido y los perfiles ~18.

4. Los siete endpoints que usa el pipeline

	ENDPOINT	HOST	PARA QUÉ
A	`/lol/league/v4/{challenger|grandmaster|master}leagues/by-queue/RANKED_SOLO_5x5`	plataforma	Semilla de jugadores de los rangos más altos. Devuelve la liga entera.
B	`/lol/league/v4/entries/RANKED_SOLO_5x5/{TIER}/{DIVISION}?page=n`	plataforma	Semilla de jugadores de Hierro a Diamante. Paginado.
C	`/lol/summoner/v4/summoners/{summonerId}`	plataforma	Resolver un identificador de jugador. Hoy no hace falta: el ranking ya devuelve el `puuid`.
D	`/lol/match/v5/matches/by-puuid/{puuid}/ids`	**región**	Identificadores de las últimas partidas clasificatorias del jugador.
E	`/lol/match/v5/matches/{matchId}`	**región**	El detalle completo de la partida. **Acá sale la variable objetivo.**
F	`/lol/league/v4/entries/by-puuid/{puuid}`	plataforma	Rango, división, puntos y victorias/derrotas del jugador.
G	`/lol/champion-mastery/v4/champion-masteries/by-puuid/{puuid}/by-champion/{championId}`	plataforma	Puntos de maestría del jugador con ese campeón concreto.

Dos aclaraciones de vocabulario:

- `puuid`: el identificador único y permanente de una cuenta de jugador. Es lo que usamos para pedir sus partidas y su perfil.
- **Cola 420**: el código que Riot le da a la modalidad *Ranked Solo/Duo*. Es la única que recolectamos, para que todas las partidas sean comparables entre sí.

> **POR QUÉ SÓLO COLA 420**
>
> Mezclar modalidades rompería la comparabilidad: una partida de ARAM se juega en otro mapa, con otras reglas y otra duración típica. Si el dataset mezclara modalidades, el modelo estaría aprendiendo a distinguir modalidades en vez de a predecir duración. Restringir a una sola modalidad es una decisión de diseño del dataset, y hay que poder defenderla.

5. Data Dragon: el catálogo público sin key

Data Dragon es el CDN público de Riot con los metadatos del juego. No pide API key y no tiene límite de pedidos. De ahí sale todo lo que el pipeline sabe de un campeón más allá de su número:

- `https://ddragon.leagueoflegends.com/api/versions.json` — la lista de versiones; la primera es la más reciente.
- `.../cdn/{version}/data/en_US/champion.json` — el catálogo de campeones.
- `.../cdn/{version}/data/en_US/summoner.json` — el catálogo de hechizos de invocador.

De cada campeón usamos: su nombre, sus **etiquetas** (Tank, Mage, Assassin, Marksman, Fighter, Support) y sus estadísticas base (ataque, defensa, magia, dificultad, vida, armadura, rango de ataque). Con eso se construyen las columnas de *composición de equipo*.

De los hechizos nos importan tres para las columnas de conteo, identificados por número: **4** Flash, **12** Teleport, **14** Ignite. Contar cuántos lleva cada equipo dice algo del estilo de juego que planea. Smite (el **11**) no se cuenta: el juego se lo impone al jungla, así que da 1 en toda partida.

> **UN DETALLE DE DISEÑO QUE CONVIENE PODER EXPLICAR**
>
> El catálogo se guarda en bronce **particionado por versión** (`ddragon/version=16.17.1/`). No es capricho: los campeones cambian entre parches, y una partida vieja tiene que poder reinterpretarse con el catálogo que le correspondía en su momento. Sin esa partición, reprocesar el bronce dentro de seis meses daría resultados distintos.

6. El recorrido de semilla a dataset

Riot **no publica un listado de partidas**. Publica un ranking de jugadores, y a cada jugador se le pueden pedir sus últimas partidas. Así que para llegar a las partidas hay que entrar por los jugadores. Ése recorrido —de un ranking a un dataset de partidas— es el corazón del pipeline:

```
por cada plataforma (la2, la1, br1, na1, euw1, kr):
  por cada tier, en ORDEN ALEATORIO (Hierro .. Challenger):
    por cada división y página:

      [A/B] pedir el ranking  ..................... la SEMILLA de jugadores

      por cada jugador del ranking:
        [D] pedir los ids de sus últimas 15 partidas clasificatorias

        por cada partida que todavía no tengamos:
          [E] pedir el detalle completo  .......... va a BRONCE

          aplicar los filtros de admisión:
             - ¿es de la Season 16?
             - ¿duró más de 5 minutos?   (descarta remakes)
             - ¿tiene 10 participantes, 5 por equipo?
          si no pasa -> anotarla como descartada y NO volver a bajarla nunca

          por cada uno de los 10 jugadores de esa partida:
            [G] pedir su maestría con el campeón que usó  .... va a BRONCE
            [F] pedir su entrada de ranking  ................. va a BRONCE
```

DOS DECISIONES DE ESTE RECORRIDO QUE HAY QUE PODER DEFENDER

POR QUÉ LOS TIERS SE RECORREN EN ORDEN ALEATORIO

Si se recorrieran siempre de Hierro hacia arriba, una corrida cortada por la mitad dejaría un dataset con puros jugadores de bronce, y el modelo aprendería el **sesgo de recolección** en vez de la señal real. Mezclando el orden, cualquier corte deja una muestra parecida al total. Con una key que vence cada 24 horas, que la corrida se corte no es hipotético: es lo normal.

LA DISTRIBUCIÓN DE RANGOS DEL DATASET NO ES LA DE LA SEMILLA

Los jugadores se muestrean recorriendo el ranking, pero de cada partida entran **los 10 participantes**, sin importar su rango. Como el emparejamiento junta gente de nivel parecido, eso no distorsiona mucho, pero es una diferencia real entre «cómo elegimos» y «qué terminamos midiendo». Es exactamente el tipo de matiz que un docente puede preguntar, y la respuesta honesta es reconocerlo.

CUÁNTO CUESTA CADA PARTIDA, MEDIDO

Medido sobre la corrida grande del 5–6/9: **283.522 pedidos para 16.908 partidas = 16,8 pedidos por partida**. El desglose es 1 pedido de detalle de partida y ~16 de perfiles y maestrías. Por eso el cuello de botella no es `match-v5` sino los endpoints de plataforma, y por eso importa tanto paralelizar por plataforma.

El número **baja a medida que el bronce crece**, porque los perfiles ya guardados no se vuelven a pedir: la plataforma que más lejos llegó (LAN, 3.334 partidas) terminó en **11,7 pedidos por partida**, contra 19,4 de la que menos avanzó. La tabla completa está en VI.4.

Ese número mejora con el volumen: un jugador que aparece en varias partidas se consulta una sola vez, porque el cliente cachea los perfiles. Cuanto más grande el dataset, más barata sale cada partida nueva.

PARTE IV

El repositorio, archivo por archivo

1. Árbol completo y criterio de organización

```
FF-15/
├── CLAUDE.md                  116 líneas · contexto e instrucciones de trabajo
├── README.md                  219 líneas · el entregable escrito
├── Dockerfile                   4 líneas · la imagen del entorno
├── requirements.txt             3 líneas · paquetes de Python
├── packages.txt                 0 líneas · paquetes de sistema (vacío)
├── pyproject.toml              32 líneas · configuración de ruff
├── .gitignore                  23 líneas · qué NO se versiona
├── .dockerignore                         · qué no entra a la imagen
├── .env                                  · PYTHONPATH (no tiene secretos)
├── airflow_settings.yaml                 · LA API KEY · gitignoreado
├── airflow_settings.example.yaml         · plantilla sin la key, versionada
│
├── .astro/                               · metadatos de Astro CLI
│   ├── config.yaml                       · nombre del proyecto
│   ├── dag_integrity_exceptions.txt      · archivos a ignorar al parsear
│   └── test_dag_integrity_default.py     · chequeo que trae Astro
│
├── .claude/skills/proyecto-integrador-ff15/
│   ├── SKILL.md                          · workflow del proyecto
│   └── references/                       · 5 documentos de referencia
│
├── dags/
│   ├── .airflowignore                    · vacío
│   └── lol_ingest.py          719 líneas · EL DAG: sólo orquestación
│
├── include/
│   ├── lol/                              · LA LÓGICA · sin importar Airflow
│   │   ├── __init__.py          9 líneas
│   │   ├── riot.py            408 líneas · cliente, pool de keys, límites
│   │   ├── ddragon.py          88 líneas · catálogo de campeones y hechizos
│   │   ├── seeds.py            77 líneas · recorrido del ranking
│   │   ├── schema.py          103 líneas · las 137 columnas y sus tipos
│   │   └── transform.py       191 líneas · JSON crudo → fila
│   └── output/                           · SALIDAS · gitignoreado
│       ├── bronze/                       · capa bronce
│       ├── silver/_parciales/            · plata por plataforma
│       ├── reports/calidad.md            · el chequeo de la consigna
│       └── lol_AAAA-MM-DD.csv            · EL ENTREGABLE
│
├── tests/dags/
│   └── test_dag_integrity.py   90 líneas · 10 tests
│
├── docs/                                 · LA DOCUMENTACIÓN INTEGRAL
│   ├── documentacion-proyecto.html       · este documento (fuente editable)
│   ├── documentacion-proyecto.pdf        · este documento (63 páginas)
│   └── generar_pdf.py                    · regenera el PDF con Chrome
```

```
│
└── Documentacion/                          · material de la cátedra
```

EL CRITERIO QUE ORDENA TODO: `DAGS/` CONTRA `INCLUDE/`

Ésta es la separación más importante del proyecto y la cátedra la subraya:

> *«El dag-processor parsea `dags/` cada pocos segundos, así que ahí va sólo la definición del flujo. La lógica pesada vive en `include/` y se importa.»*

Nosotros llevamos esa idea un paso más allá, con una regla dura: `include/lol/` **no importa Airflow en ninguna línea.**

VENTAJA	POR QUÉ
Se puede testear sin Airflow	Los módulos se importan con `python` a secas y se prueban con `pytest` plano.
Se reusa en las Unidades 2, 3 y 4	En un notebook de exploración no hay un DAG alrededor, pero `from lol import schema` funciona igual.
El parseo del DAG es rápido	Nada pesado se importa al tope del archivo del DAG.

La frontera es estricta: si un módulo de `include/lol/` necesitara un valor de configuración de Airflow, se lo pasa la tarea por parámetro. Por eso `RiotClient` recibe las keys como argumento en vez de ir a buscarlas él mismo.

2. Configuración del entorno

`DOCKERFILE`

```
FROM astrocrpublic.azurecr.io/runtime:3.3-4

# El código del proyecto vive en include/lol y se importa como `lol`.
ENV PYTHONPATH="/usr/local/airflow/include:${PYTHONPATH}"
```

Sólo dos instrucciones. La primera fija la imagen base: **Astro Runtime 3.3-4**, que trae Airflow 3.3 ya armado — la misma versión que usa la cátedra. La segunda agrega `include/` a la ruta de búsqueda de Python, que es lo que permite escribir `from lol import schema` desde el DAG.

> UNA DIFERENCIA CON EL PROYECTO DE LA CÁTEDRA
>
> El `Dockerfile` de `airflow-fifa` instala Chromium con Playwright, porque scrapea un sitio protegido por Cloudflare. El nuestro no lo necesita: Riot es una API REST y se responde con pedidos HTTP comunes. Eso hace que nuestra imagen sea **alrededor de 1 GB más liviana** y que el primer arranque sea mucho más rápido.

`REQUIREMENTS.TXT`

```
pandas>=2.2
requests>=2.31
apache-airflow-providers-http>=6.0
```

Tres dependencias. `pandas` para armar y escribir los CSV, `requests` para hablar con la API, y el provider de HTTP para que exista el tipo de conexión `http` donde guardamos la key.

Importante: si se toca este archivo hay que hacer `astro dev restart`, porque los paquetes se instalan al construir la imagen. Cambiar un `.py` no necesita restart.

`PYPROJECT.TOML`

Configura **ruff**, el linter y formateador de Python. El proyecto de la cátedra no trae ninguno: el estilo se sostiene sólo por convención. Con un grupo de varias personas tocando los mismos archivos eso se degrada rápido, así que acá está por escrito: línea de 79 caracteres, orden de imports con `lol` reconocido como paquete propio, y un conjunto de reglas activas.

`.ASTRO/`

Metadatos que usa Astro CLI. `config.yaml` tiene el nombre del proyecto; `test_dag_integrity_default.py` es un chequeo que trae Astro y que corre con `astro dev parse`.

3. Credenciales y seguridad

Ésta es la parte donde el proyecto se juega algo serio, y donde hay un antecedente concreto que conviene conocer.

> EL ANTECEDENTE
>
> El prototipo previo del grupo (la carpeta `FF15`, sin guion) tenía **cinco API keys de Riot escritas en texto plano dentro de un archivo `.py`**, y ese repositorio está publicado en GitHub — con las keys también en el historial de commits, así que borrarlas del archivo no alcanza para sacarlas. Las development keys vencen cada 24 horas, así que el daño real fue acotado, pero el patrón es exactamente lo que no hay que hacer.

La cátedra lo dice literal en el apunte de Airflow: *«si tu fuente pide API key, token o usuario y contraseña, esa credencial va en una Connection, nunca en el `.py`»*.

CÓMO LO RESOLVIMOS

La key vive en una **Connection de Airflow** llamada `riot_api`, declarada en `airflow_settings.yaml`:

```
airflow:
  connections:
    - conn_id: riot_api
      conn_type: http
      conn_host: api.riotgames.com
      conn_schema: https
      conn_password: RGAPI-xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

Astro CLI la carga sola al arrancar, así que no hay que darla de alta a mano en la interfaz. El código la lee así, y ésta es la única función de todo el proyecto que toca la credencial:

```
def api_keys() -> list[str]:
    """Las keys, desde la Connection. Nunca desde el código."""
    conn = BaseHook.get_connection(CONN_ID)
    crudo = conn.password or ""
    keys = [k.strip() for k in crudo.split(",") if k.strip()]
    if not keys:
        raise ValueError(
            f"La Connection `{CONN_ID}` no tiene ninguna API key ...")
    return keys
```

El `split(",")` es lo que habilita el **pool**: se pueden pegar varias keys separadas por coma —una por integrante— y el cliente las usa en paralelo.

LOS DOS ARCHIVOS, Y POR QUÉ SON DOS

ARCHIVO	¿SE VERSIONA?	POR QUÉ
`airflow_settings.yaml`	**NO**	Tiene la key real. Está en `.gitignore`.
`airflow_settings.example.yaml`	**SÍ**	Plantilla con un placeholder. Es lo que copia un integrante nuevo al clonar el repo.

> **UNA DIFERENCIA DELIBERADA CON EL PROYECTO DE LA CÁTEDRA**
>
> En `airflow-fifa` el archivo `airflow_settings.yaml` **sí** se versiona, y el comentario de su `.gitignore` explica por qué: su única conexión es la de la PokéAPI, que es pública y no tiene ningún secreto. En el nuestro hay una credencial real, así que la decisión se invierte. Saber *por qué* nuestro caso difiere del ejemplo es justamente el tipo de cosa que distingue copiar de entender.

4. `include/lol/` — la lógica del dominio

4.1 `RIOT.PY` · 408 LÍNEAS · EL CLIENTE DE LA API

Es el módulo más complejo del proyecto. Su responsabilidad: hablar con la API de Riot respetando los límites, rotando entre varias keys y sin que nadie de afuera tenga que pensar en eso.

Las constantes del dominio — el mapeo de plataformas, los rangos y los límites:

```
PLATFORM_ROUTING = {
    "la2": {"region": "americas", "name": "LAS"},
    "la1": {"region": "americas", "name": "LAN"},
    "br1": {"region": "americas", "name": "BR"},
    "na1": {"region": "americas", "name": "NA"},
    "euw1": {"region": "europe", "name": "EUW"},
    "kr": {"region": "asia", "name": "KR"},
}

ALL_TIERS = ["IRON", "BRONZE", "SILVER", "GOLD", "PLATINUM", "EMERALD",
             "DIAMOND", "MASTER", "GRANDMASTER", "CHALLENGER"]

# Los tres de arriba no tienen divisiones y su endpoint devuelve la liga
# entera: no acepta `page`.
APEX_TIERS = {"MASTER", "GRANDMASTER", "CHALLENGER"}

PER_SECOND_LIMIT = 20      ; PER_SECOND_WINDOW = 1.0
PER_TWO_MIN_LIMIT = 100    ; PER_TWO_MIN_WINDOW = 120.0
```

El limitador de velocidad es el corazón del módulo. La idea: se lleva la cuenta de los pedidos hechos por cada par *(key, host)*, y antes de disparar uno nuevo se verifica que haya cupo en las dos ventanas de tiempo.

```
def _tomar_turno(self, host: str, key: str | None = None) -> str:
    """Devuelve una key con cupo para `host`, esperando si hace falta.

    No castiga a una key en un host donde sí tiene cupo sólo porque
    está en cooldown en otro: los baldes son independientes.

    Si `key` viene dada, no rota: espera cupo en ESA key puntual. Es
    el caso de un pedido sobre un puuid (ver más abajo).
    """
    while True:
        ahora = time.time()
        esperas = []
        with self._lock:
            if key is not None:
                if key in self._invalidas:
                    raise _KeyDueñaInvalida(key)
                candidatas = [key]
            else:
                candidatas = [k for k in self.keys if k not in self._invalidas]
                if not candidatas:
                    raise RiotKeysExhausted(...)
            for _ in range(len(candidatas)):
                self._idx = (self._idx + 1) % len(candidatas)
                k = candidatas[self._idx]
                # ¿está esta key castigada en este host?
                # ¿cuántos pedidos lleva en los últimos 120 s?
                # ¿y en el último segundo?
                # si hay cupo: se anota el pedido y se devuelve la key
        # si ninguna tenía cupo, dormir lo mínimo necesario
        time.sleep(min(max(min(esperas), 0.02), 2.0))
```

POR QUÉ EL CONTROL ES *PROACTIVO* Y NO REACTIVO

La forma fácil sería mandar pedidos hasta que Riot conteste `429 Too Many Requests` y ahí frenar. Funciona, pero desperdicia un pedido y el tiempo de ida y vuelta *cada vez*. Nuestro cliente calcula antes de disparar si hay cupo, y sólo espera lo justo. El `429` queda igual como red de contención por si Riot aplica un límite que no conocemos.

UN `PUUID` SÓLO LO PUEDE LEER LA KEY QUE LO EMITIÓ

Riot devuelve cada `puuid` **encriptado para la key puntual** que hizo esa request. Pedir algo con él usando otra key del pool —aunque esa otra key sea perfectamente válida— da `400 Bad Request - Exception decrypting <puuid>`, un mensaje que no sugiere la causa real: parece un dato corrupto, no una key equivocada.

Por eso `_tomar_turno` tiene el parámetro `key`: los pedidos D, F y G (los que llevan un `puuid` en la URL) **no rotan**, van con la key que trajo ese `puuid` — la de `league_entries` para el jugador semilla, o la de `match` para los 10 participantes de una partida (esos `puuid` vienen embebidos en el detalle de la partida, encriptados para la key que pidió *esa* partida, no para la de la semilla). Si esa key venció mientras tanto, no hay fallback posible: se salta el pedido en vez de gastar reintentos contra keys que nunca van a poder leerlo.

Se encontró corriendo el pipeline con el pool completo: las seis plataformas fallaban con `400` desde el primer pedido, con las siete keys por igual. Se confirmó pegándole a la API con la misma URL y distintas keys: una respondía `200`, las demás `400 Exception decrypting`. El detalle completo está en las trampas conocidas de `.claude/skills/proyecto-integrador-ff15/references/riot-api.md`.

El manejo de errores, que traduce cada código HTTP a una acción distinta:

CÓDIGO	QUÉ SIGNIFICA	QUÉ HACE EL CLIENTE
`200`	Todo bien	Devuelve el JSON (y la key que respondió, para poder heredarla en pedidos siguientes sobre el mismo puuid).
`429`	Demasiados pedidos	Lee el encabezado `Retry-After`, castiga esa key *en ese host* y sigue con otra key.
`401` / `403`	Key inválida o vencida	La saca del pool. Si se vacía el pool, lanza `RiotKeysExhausted` con un mensaje que dice exactamente qué hacer.
`400` con key dueña de un puuid	Esa key venció o quedó inválida entre medio	No hay otra key que sirva para ese puuid: se salta el pedido en vez de reintentar.
`404`	No existe	Devuelve `None` sin reintentar.
`5xx` / timeout	Problema del servidor o de red	Reintenta con espera creciente.

Las cachés en memoria. Un jugador aparece en muchas partidas; sin caché, cada partida volvería a pedir los mismos 10 perfiles. Con ella, el costo por partida baja a medida que el dataset crece.

Las dos funciones puras del final traducen las respuestas crudas a valores del dataset. Están en este módulo pero no dependen de la red, y eso es lo que permite que la capa plata las use leyendo del bronce:

```
def resumen_liga(entradas):
    """De las entradas de liga crudas al elo estimado y el winrate."""
    for e in (entradas or []):
        if e.get("queueType") != "RANKED_SOLO_5x5":
            continue          # un jugador puede tener también flex:
                              # mezclarlas compararía cosas distintas
        ...
        return {"estimated_elo": TIER_BASE_MMR[tier] + RANK_BONUS[rank]
                                 + int(lp * 0.5),
                "winrate": round(wins / total, 4),
                "es_unranked": 0}
    return {"estimated_elo": 1500, "winrate": 0.50, "es_unranked": 1}
```

4.2 `DDRAGON.PY` · 88 LÍNEAS · EL CATÁLOGO DEL JUEGO

Descarga e indexa los metadatos de campeones y hechizos. Está separado del cliente de Riot a propósito: Data Dragon es público, no pide key y no tiene límite, así que mezclarlo con el pool de keys sería gastar cupo en algo que no lo necesita.

La función clave reindexa el catálogo:

```
def index_champions(champion_json: dict) -> dict[int, dict]:
    """Indexa por id numérico, que es como los devuelve match-v5.

    El CDN los publica indexados por nombre interno ("MonkeyKing"), pero
    las partidas traen `championId`. Sin este reindexado habría que hacer
    una búsqueda lineal por cada uno de los 10 campeones de cada partida.
    """
```

4.3 `SEEDS.PY` · 77 LÍNEAS · EL RECORRIDO DEL RANKING

Un **generador** que va entregando jugadores del ranking. Es un generador y no una lista por una razón concreta: la corrida se corta cuando se alcanza la cuota de partidas, no cuando se agota el ranking. Pedir todas las páginas por adelantado gastaría cupo en jugadores que no se van a visitar nunca.

```
def iter_puuids(client, platform, tiers, max_paginas=40, semilla=None):
    rnd = random.Random(semilla)
    vistos: set[str] = set()

    for page in range(1, max_paginas + 1):
        tiers_mezclados = list(tiers)
        rnd.shuffle(tiers_mezclados)          # el orden aleatorio
        for tier in tiers_mezclados:
            divisiones = ["I"] if tier in APEX_TIERS else DIVISIONS
            if tier in APEX_TIERS and page > 1:
                continue                       # su endpoint no pagina
            for division in divisiones:
                entradas, owner_key = client.league_entries(
                    platform, tier, division, page)
                rnd.shuffle(entradas)
                for e in entradas:
                    puuid = e.get("puuid")
                    if not puuid or puuid in vistos:
                        continue
                    vistos.add(puuid)
                    yield puuid, tier, owner_key   # la key dueña del puuid
```

El `owner_key` viaja con cada `puuid` hasta `land_bronze`, que lo usa para `match_ids`. Y cuando `match()` trae el detalle de una partida, la key que respondió esa llamada se vuelve la dueña de los `puuid` de los 10 participantes, para `league_by_puuid` / `mastery`. Ver el aviso sobre keys y puuids en 4.1.

4.4 `SCHEMA.PY` · 103 LÍNEAS · EL CONTRATO DEL DATASET

Define **qué columnas tiene el dataset y en qué orden**. Es el contrato: si el transformador produce algo distinto, la validación falla y el DAG falla.

```
EQUIPOS = ["t1", "t2"]
POSICIONES = ["top", "jng", "mid", "bot", "sup"]

FEATURES_JUGADOR = [        # 10 por jugador
    "champ_id", "champ_name", "sum1", "sum2",
    "rune_primary", "rune_sub",
    "player_elo", "player_winrate", "champ_mastery_pts", "is_main",
]

FEATURES_EQUIPO = [ ... ]   # 17 por equipo

def columnas_equipo(prefijo):
    """Las 50 columnas de jugador + 17 de equipo, para t1 o t2."""
    cols = [f"{prefijo}_{pos}_{f}"
            for pos in POSICIONES for f in FEATURES_JUGADOR]
    cols += [f"{prefijo}_{f}" for f in FEATURES_EQUIPO]
    return cols

COLUMNS = (IDENTIDAD + TARGET + CALIDAD
           + columnas_equipo("t1") + columnas_equipo("t2"))
```

Las columnas se **generan**, no se escriben a mano. Así es imposible que se escape un typo en `t2_sup_champ_mastery_pts`, y agregar una variable por jugador es cambiar una lista de diez elementos en vez de escribir veinte nombres.

La cuenta: `3 + (10 × 5 × 2) + (17 × 2) = 3 + 100 + 34 = 137`.

El módulo también fija las **reglas de admisión**:

```
SEASON_PREFIX = "16."
DURACION_MINIMA_S = 300               # descarta remakes
DURACION_MAXIMA_PLAUSIBLE_S = 20_000  # ~5,5 hs: nada dura tanto
```

4.5 `TRANSFORM.PY` · 191 LÍNEAS · DE JSON CRUDO A FILA

Todo este módulo es **puro**: no toca la red ni el disco. Recibe el JSON tal como lo guardó el bronce más los perfiles de los jugadores, y devuelve un diccionario con las 137 columnas. Esa pureza es lo que permite corregir un error de parseo y reprocesar todo el bronce sin gastar un solo pedido.

La corrección de la unidad de tiempo — una trampa real de esta fuente:

```
def normalizar_duracion(bruto) -> int:
    """`gameDuration` a segundos.

    Riot cambió el criterio alrededor del parche 11.20: las partidas con
    `gameEndTimestamp` lo devuelven en segundos y las anteriores en
    milisegundos. Mezclar las dos unidades en la misma columna arruinaría
    el target sin que nada falle, que es la peor clase de bug.
    """
    valor = int(bruto or 0)
    if valor > schema.DURACION_MAXIMA_PLAUSIBLE_S:
        log.info("gameDuration=%s parece venir en ms; se convierte a s.", valor)
        valor = valor // 1000
    return valor
```

Los filtros de admisión, que lanzan una excepción con el motivo exacto:

```
def admitir(match_data: dict):
    info = match_data.get("info") or {}
    version = info.get("gameVersion") or ""
    if not version.startswith(schema.SEASON_PREFIX):
        raise PartidaDescartada(f"no es Season 16 (gameVersion={version!r})")
    duracion = normalizar_duracion(info.get("gameDuration"))
    if duracion < schema.DURACION_MINIMA_S:
        raise PartidaDescartada(f"dura {duracion}s, parece un remake")
    participantes = info.get("participants") or []
    if len(participantes) != 10:
        raise PartidaDescartada(f"tiene {len(participantes)} participantes, no 10")
    for team_id in (100, 200):
        n = sum(1 for p in participantes if p.get("teamId") == team_id)
        if n != 5:
            raise PartidaDescartada(f"el equipo {team_id} tiene {n} jugadores, no 5")
    return info, participantes, duracion
```

La asignación de carriles, y una corrección importante respecto del prototipo previo:

```
def ordenar_por_rol(participantes):
    """Ordena los 5 de un equipo en top/jng/mid/bot/sup.

    Devuelve también CUÁNTOS quedaron sin emparejar. `teamPosition`
    puede venir vacío en remakes y en partidas con autofill; en ese caso
    el jugador se coloca igual en el slot que sobre —descartar la partida
    entera sería peor— pero el conteo viaja hasta la columna
    `roles_inferidos`, así esas filas se pueden filtrar más adelante en
    vez de arrastrar una etiqueta silenciosamente equivocada.
    """
```

UN BUG DEL PROTOTIPO PREVIO, CORREGIDO

El prototipo hacía exactamente lo mismo **sin contar nada**. Cuando la fuente no informaba la posición, el jugador terminaba etiquetado como «jungla» sin haberlo sido, y no quedaba ningún rastro. El dato estaba mal y no había forma de saberlo. Ahora la columna `roles_inferidos` registra cuántos jugadores de esa partida quedaron sin emparejar, y las filas afectadas se filtran con `df[df.roles_inferidos == 0]`. **El error existía igual; lo que cambió es que ahora es visible.**

5. `dags/lol_ingest.py` — la orquestación

719 líneas, pero la mayoría son docstrings: cada tarea explica *por qué* está donde está. Esos textos se renderizan en la interfaz de Airflow y son la chuleta para la defensa oral.

LA DECLARACIÓN DEL DAG

```
@dag(
    dag_id="lol_ingest",
    schedule=None,
    start_date=pendulum.datetime(2026, 8, 1,
                                 tz="America/Argentina/Buenos_Aires"),
    catchup=False,
    max_active_tasks=6,
    tags=["ciencia-de-datos", "unidad-1", "integrador", "5K09-02"],
    doc_md=__doc__,
    params={ ... },
)
def lol_ingest():
    ...

lol_ingest()   # SIN ESTA LÍNEA el DAG no aparece en la interfaz
```

ARGUMENTO	QUÉ SIGNIFICA
`schedule=None`	Se dispara a mano, no automáticamente. Justificado abajo.
`catchup=False`	Si el entorno estuvo apagado una semana, no intenta recuperar las corridas perdidas.
`max_active_tasks=6`	Las seis plataformas corren a la vez.
`doc_md=__doc__`	El docstring del archivo se muestra en la interfaz.

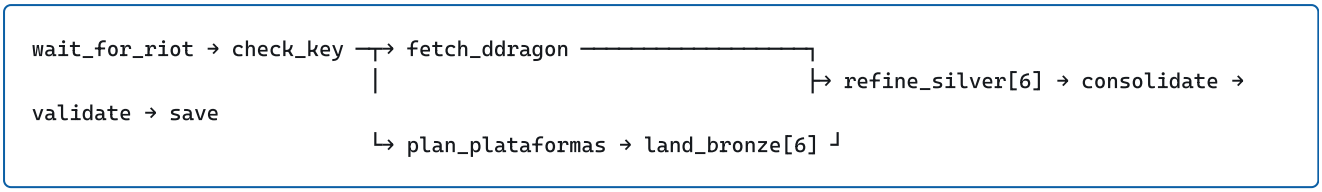
> **POR QUÉ `SCHEDULE=NONE` Y NO `@DAILY`**
>
> El DAG de la cátedra corre todos los días. El nuestro no, y la razón es concreta: la development key de Riot **vence cada 24 horas**. Un DAG programado corriendo sin nadie mirando fallaría todos los días apenas venciera la key, y el ruido de esas fallas taparía las reales. Se dispara a mano, con la key recién renovada. Es una decisión defendible y hay que saber explicarla: el DAG de ejemplo de la cátedra (`pokemon_api`) también usa `schedule=None`.

LOS CUATRO PARÁMETROS DE CORRIDA

PARÁMETRO	DEFAULT	QUÉ HACE
`partidas_objetivo`	1500	Cuántas partidas juntar. Se reparte en partes iguales entre las plataformas.
`plataformas`	las 6	Qué servidores recorrer.
`tiers`	los 10	De qué rangos salen los jugadores semilla.
`forzar`	`false`	Ignora el bronce ya bajado y vuelve a pedir todo.

Todos llevan `title` y `description` en castellano, para que el formulario de disparo de la interfaz se explique solo.

EL GRAFO



LAS NUEVE TAREAS, UNA POR UNA

1 · `wait_for_riot` — un *sensor*. Comprueba que la API responde y que la key sirve. Sondea cada 2 minutos hasta 30, en modo `reschedule` para no ocupar un trabajador durmiendo, y con `soft_fail=True` para que agotar el tiempo deje la tarea en *salteada* y no en *fallada*: que Riot no responda no es un error de nuestro pipeline.

Distingue dos fallas que se resuelven distinto, y eso es lo que la hace útil:

```
try:
    responde = cliente.ping()
except RiotKeysExhausted as e:
    # Ninguna key del pool sirve. Cortar ya, no seguir sondeando.
    return PokeReturnValue(is_done=True,
                           xcom_value={"ok": False, "motivo": str(e)})

if responde:
    return PokeReturnValue(is_done=True, xcom_value={"ok": True})

log.warning("Riot no responde todavía. Reintento en 2 min.")
return PokeReturnValue(is_done=False)
```

> VERIFICADO EN LA PRÁCTICA
>
> Con una key inválida, esta tarea resuelve **en 3 segundos** en vez de sondear media hora. Si Riot está caído, en cambio, espera los 30 minutos completos, porque un corte de diez minutos no tiene por qué arruinar la corrida. *Esperar arregla un corte; no arregla una key vencida.*

2 · `check_key` — un *cortocircuito*. Si el sensor no consiguió respuesta, devuelve `False`, todo lo de abajo queda salteado y la corrida termina **bien**. Corre con `trigger_rule=ALL_DONE` porque su tarea anterior puede haber quedado salteada — que es justamente el caso que tiene que manejar. Con la regla por defecto no se ejecutaría nunca cuando más falta hace.

3 · `fetch_ddragon` — primera tarea de **bronce**. Baja el catálogo de campeones y hechizos y lo guarda crudo, particionado por versión. Si ya está, no lo vuelve a pedir.

4 · `plan_plataformas` — reparte el objetivo entre las plataformas elegidas:

```
cuota = math.ceil(params["partidas_objetivo"] / len(plataformas))
```

> POR QUÉ SE REPARTE POR ADELANTADO
>
> Airflow corre cada tarea mapeada en **su propio proceso**, así que no existe un contador compartido en memoria como tendría un script monolítico. La cuota se reparte antes de arrancar y se deduplica al final. Es un ejemplo lindo de cómo la arquitectura del orquestador condiciona el diseño del algoritmo.

5 · `land_bronze` — **bronce**. Se mapea sobre las plataformas, así que en la interfaz aparecen seis instancias. **Es la única tarea de todo el DAG que toca la red.** No parsea, no limpia, no valida: guarda el JSON tal como vino y anota dónde quedó.

El detalle más valioso de esta tarea es el orden de las operaciones:

```
datos = cliente.match(region, match_id)

# Se filtra ANTES de gastar ~17 pedidos en los perfiles de los
# 10 jugadores de una partida que igual se va a tirar.
try:
    _, participantes, _ = transform.admitir(datos)
except transform.PartidaDescartada as e:
    descartadas.add(match_id)      # y NO se vuelve a bajar nunca
    continue

escribir_json(destino, datos)

with ThreadPoolExecutor(max_workers=4) as pool:
    list(pool.map(perfil_de, participantes))
```

Además, la escritura es **atómica**: se escribe a un archivo temporal y se renombra. Así una tarea cortada a mitad de escritura no deja un archivo truncado que reviente el parseo después.

6 · `refine_silver` — **plata**. Una por plataforma, y **sin red**. Convierte el JSON crudo en filas tipadas leyendo del bronce.

> UN BUG QUE ENCONTRAMOS Y CORREGIMOS, Y QUE VALE LA PENA CONOCER
>
> La primera versión iteraba sólo las partidas que había bajado *esa* corrida. Con eso, correr el DAG una segunda vez nunca hubiera hecho crecer el CSV: cada corrida producía un archivo con su propia cuota y pisaba el anterior. Toda la estrategia de acumular hacia 20.000 en varias tandas no funcionaba. Ahora hace `glob` sobre **todo el bronce** de su plataforma:
>
> ```
> rutas = sorted(
>     (BRONCE_DIR / "matches" / f"plataforma={platform}").glob("*.json.gz"))
> ```
>
> Verificado a escala real el 6/9: con **19.133 partidas ya en bronce**, una corrida de cuota 2 por plataforma bajó 12 partidas nuevas y produjo un CSV de **19.145 filas** — el acumulado entero, no las 12 de esa corrida. Es la propiedad que permite regenerar el entregable en 15 minutos sin volver a recolectar.

7 · `consolidate` — junta los parciales de las seis plataformas, **deduplica por** `match_id` y castea los tipos. La deduplicación acá es lo que sostiene la promesa de que la clave primaria es única, que es el criterio que más pesa en la entrega.

8 · `validate` — los chequeos duros. Si alguno falla, el DAG falla: no se publica un dataset roto. Verifica que las columnas sean exactamente las esperadas, que la clave no repita, que no haya nulos en las columnas obligatorias, que el objetivo esté en rango plausible y que ninguna columna esté 100 % vacía.

Y además escribe el **reporte de calidad** en `include/output/reports/calidad.md`, con los seis números que la consigna pide tener a mano — para no calcularlos a las apuradas cinco minutos antes de la entrega.

9 · `save` — escribe el entregable fechado. Un detalle sutil:

```
dag_run = context["dag_run"]
momento = dag_run.logical_date or dag_run.run_after
ds = momento.date().isoformat()
```

La fecha sale del *DagRun* y no de la variable `ds`. `ds` sólo existe cuando el DAG tiene `schedule`, y el nuestro corre a demanda. Es un tropiezo clásico al pasar de Airflow 2 a 3, y está explícitamente listado entre los errores frecuentes del apunte de la cátedra.

6. Tests, calidad de código y documentación

`TESTS/DAGS/TEST_DAG_INTEGRITY.PY` · **10 TESTS**

GRUPO	QUÉ VERIFICA
El DAG	Que no haya errores de import, que se registre, que estén las nueve tareas, que corra a demanda y que no haga catchup.
El esquema	Que sean 137 columnas sin repetidas, que cada columna caiga en exactamente un grupo de tipo, y que estén la clave y el objetivo.
Las trampas de la fuente	Que una duración en milisegundos se normalice, y que un `teamPosition` vacío se cuente en `roles_inferidos`.

Los dos últimos son los más valiosos: son *tests de regresión* sobre bugs reales. Si alguien en el futuro simplifica esa lógica sin entenderla, el test lo frena.

> **UN DETALLE TÉCNICO QUE COSTÓ DESCUBRIR**
> En Airflow 3, `DagBag.get_dag()` consulta la tabla `dag` de la base de metadatos, que no existe en el contexto de pytest. Hay que usar el diccionario `DagBag.dags`, que es el resultado directo del parseo. Vale la pena saberlo: los tests del proyecto de la cátedra (`airflow-fifa`) **fallan**, en parte por esto y en parte porque quedaron desactualizados respecto de su propio DAG.

DOCUMENTACIÓN DEL REPOSITORIO

ARCHIVO	PARA QUÉ
`README.md`	El entregable escrito: qué es el dataset, cómo se ejecuta, y las decisiones que hay que poder defender.
`CLAUDE.md`	Contexto e instrucciones para trabajar en el repo con asistencia de IA. Incluye la regla de mantener este documento actualizado.
`.claude/skills/`	La skill del proyecto: el workflow y cinco documentos de referencia (entregas, criterios, convenciones de Airflow, la API de Riot, el esquema).
`docs/`	Este documento y el script que regenera el PDF.
`Documentacion/`	El material original de la cátedra. Es la fuente de verdad de las consignas.

PARTE V

El dataset producido

1. Unidad de análisis, clave y objetivo

Una fila es	una partida clasificatoria de League of Legends, modalidad Solo/Duo (cola 420), de la Season 16
Clave primaria	`match_id` — por ejemplo `LA2_1617259431`
Columna objetivo	`game_duration_s` — duración en segundos, continua
Tipo de problema	Regresión (el objetivo es un número continuo, no una categoría)
Ancho	137 columnas

El `match_id` lleva el prefijo de la plataforma donde se jugó (`LA2_`, `KR_`, `EUW1_`...), lo cual tiene un efecto lateral útil: la clave es única a nivel global sin necesidad de combinarla con nada, y de paso permite saber de qué servidor vino cada partida con un simple `df.match_id.str.split("_").str[0]`.

2. Las 137 columnas por bloques

La cuenta: `3 + (10 × 5 × 2) + (17 × 2) = 137`.

BLOQUE A · IDENTIDAD, OBJETIVO Y CALIDAD — 3 COLUMNAS

COLUMNA	TIPO	DE DÓNDE SALE
`match_id`	texto	`metadata.matchId` del detalle de partida
`game_duration_s`	entero	`info.gameDuration`, con la corrección de milisegundos
`roles_inferidos`	entero	**Derivada.** Cuántos de los 10 jugadores no traían su carril informado

BLOQUE B · POR JUGADOR — 100 COLUMNAS

Diez variables × cinco posiciones × dos equipos. El nombre se arma como `{t1|t2}_{top|jng|mid|bot|sup}_{variable}`. Por ejemplo: `t1_top_player_elo` es el elo del jugador que ocupó el carril superior del equipo azul.

VARIABLE	TIPO	QUÉ ES Y DE DÓNDE SALE
`champ_id`	entero	Número del campeón elegido, de la partida.
`champ_name`	texto	Nombre del campeón. Es la única categórica de texto del dataset.
`sum1`, `sum2`	entero	Los dos hechizos de invocador (4 Flash, 11 Smite, 12 Teleport, 14 Ignite…).
`rune_primary`	entero	Rama principal de runas. `0` si el payload no la trae.
`rune_sub`	entero	Rama secundaria. `0` si falta.
`player_elo`	entero	**Derivada.** Proxy de habilidad. Ver 5.4.
`player_winrate`	flotante	Victorias sobre total en solo/duo. `0.50` si no tiene ranking.
`champ_mastery_pts`	entero	Puntos de maestría con ese campeón. `0` si nunca lo jugó.
`is_main`	entero	**Derivada.** `1` si la maestría supera 50.000 puntos.

BLOQUE C · AGREGADOS DE EQUIPO — 34 COLUMNAS

Diecisiete variables × dos equipos, con el nombre `{t1|t2}_{variable}`. Describen la *composición* del equipo: qué tipo de equipo se armó en el draft.

VARIABLE	CÓMO SE CALCULA
`mains_count`	Cuántos de los 5 jugadores usan un campeón que dominan (suma de `is_main`).
`num_tanks`, `num_mages`, `num_assassins`, `num_marksmen`, `num_fighters`, `num_supports`	Cuántos de los 5 campeones tienen esa etiqueta en Data Dragon.
`avg_attack`, `avg_defense`, `avg_magic`, `avg_difficulty`, `avg_attackrange`	Promedio de esa estadística base sobre los 5 campeones.
`total_base_hp`, `total_base_armor`	Suma cruda (no promedio) de vida y armadura base.
`count_teleport`, `count_ignite`, `count_flash`	Cuántos de los 10 hechizos del equipo son ése.

> **UN DETALLE QUE PUEDE PREGUNTAR EL DOCENTE**
>
> Las seis columnas de conteo por tipo de campeón **no suman 5**. Un campeón puede tener varias etiquetas a la vez —Ornn es Tank *y* Fighter—, así que la suma puede dar más de 5. No es un error de cálculo: es cómo Data Dragon clasifica a los campeones, y saberlo evita interpretar mal esas columnas en la exploración de la Unidad 2.

POR QUÉ NO HAY UNA COLUMNA `COUNT_SMITE`

El esquema del prototipo traía `t1_count_smite` y `t2_count_smite`. En toda la muestra valían **1**: Smite es obligatorio para el jungla y hay un jungla por equipo. Una columna constante no aporta nada a un modelo —varianza cero— así que se quitó. Los otros tres conteos de hechizo (`flash`, `teleport`, `ignite`) sí varían y se conservan.

3. Reglas de admisión de una partida

Una partida entra al dataset sólo si pasa los cuatro filtros. Si no pasa, se anota en el índice de descartadas y **no se vuelve a descargar nunca**.

FILTRO	REGLA	POR QUÉ
Temporada	`gameVersion` empieza con `"16."`	Las reglas del juego cambian entre temporadas. Mezclarlas compararía cosas distintas.
Duración mínima	≥ 300 segundos	Descarta los *remakes*: partidas que se cancelan en los primeros minutos porque alguien no se conectó. No son partidas reales.
Participantes	Exactamente 10	Un payload con otra cantidad está corrupto o es de otra modalidad.
Equipos	5 y 5	Garantiza que se puedan armar los dos bloques de cinco jugadores.

El filtro se aplica **antes** de pedir los perfiles de los 10 jugadores. Es una decisión de eficiencia: ahorra ~17 pedidos por cada partida que igual se iba a descartar.

4. Variables derivadas y su justificación

Tres columnas no vienen de la fuente: las calculamos nosotros. Hay que poder defender las tres.

`PLAYER_ELO` — EL PROXY DE HABILIDAD

> **RIOT NO EXPONE EL MMR**
>
> El *matchmaking rating* real —el número interno con el que Riot empareja jugadores— es privado. Lo que sí publica es el rango visible: tier, división y puntos de liga. `player_elo` es una **construcción del grupo** que convierte esa escala ordinal en un número comparable. Hay que presentarla como tal: es una variable derivada, no observada.

```
estimated_elo = TIER_BASE_MMR[tier] + RANK_BONUS[rank] + int(lp * 0.5)
```

TIER	BASE	TIER	BASE	DIVISIÓN	BONUS
IRON	800	EMERALD	2300	IV	0
BRONZE	1100	DIAMOND	2600	III	75
SILVER	1400	MASTER	2900	II	150
GOLD	1700	GRANDMASTER	3200	I	225
PLATINUM	2000	CHALLENGER	3500		

Ejemplo verificado: un jugador Oro I con 40 puntos da `1700 + 225 + 20 = 1945`.

La lógica de los coeficientes: 300 puntos entre tiers consecutivos, repartidos en cuatro divisiones de 75, y los puntos de liga pesan la mitad para que un jugador a punto de ascender quede cerca del tier siguiente sin superponerse. **Son una elección del grupo, no una medición.** Si en la Unidad 3 resulta que la

escala no ayuda al modelo, se puede reemplazar por el tier como categórica — el bronce guarda la respuesta cruda, así que rehacerlo no cuesta ningún pedido a la API.

`IS_MAIN` — EL UMBRAL DE 50.000 PUNTOS

Marca si el jugador domina el campeón que eligió. El umbral lo eligió el grupo: 50.000 puntos de maestría equivalen aproximadamente a decenas de partidas con ese campeón. Es un corte arbitrario y conviene decirlo así. Una alternativa para la Unidad 2 sería usar directamente `champ_mastery_pts` (que ya está en el dataset) y dejar que el modelo encuentre el corte.

`ROLES_INFERIDOS` — LA BANDERA DE CALIDAD

Cuenta cuántos de los 10 jugadores de esa partida no traían su carril informado en la fuente y fueron asignados al slot que sobraba. Filas con `roles_inferidos > 0` tienen todos sus datos correctos; lo único incierto es la etiqueta de carril. Se filtran con `df[df.roles_inferidos == 0]`.

5. Los nulos: dónde, por qué y qué se hizo

La consigna es explícita: *«No pedimos cero nulos: eso no existe en datos reales. Pedimos que sepas cuáles tenés y qué significan.»*

En nuestro dataset **no hay nulos**, y eso no es porque los datos vengan perfectos: es porque donde la fuente no tiene el dato, imputamos un valor que *significa algo* en vez de dejar un hueco. Hay que poder defender esa decisión, incluyendo sus contras.

SITUACIÓN	QUÉ QUEDA	RAZONAMIENTO
Jugador sin ranking en solo/duo (nuevo, inactivo, o sólo juega flex)	`player_elo = 1500` `player_winrate = 0.50`	Codifica «habilidad desconocida, asumimos promedio». 1500 cae entre Gold y Platinum, que es la zona más poblada.
Jugador que nunca jugó ese campeón	`champ_mastery_pts = 0` `is_main = 0`	Es literalmente cierto: tiene cero puntos de maestría. No es una imputación, es el valor real.
`perks` ausente en el payload	`rune_primary = rune_sub = 0`	`0` no es un identificador de rama válido, así que el caso queda distinguible.
Campeón más nuevo que el catálogo de Data Dragon guardado	Aporta 0 a los agregados de composición	Se emite un aviso en el log para que no pase desapercibido.

> **LA CONTRA DE ESTA DECISIÓN, Y HAY QUE DECIRLA**
>
> Imputar con 1500 y 0.50 mezcla dos cosas distintas: un jugador que *es* de nivel medio y un jugador del que *no sabemos nada*. El modelo no puede distinguirlos.
>
> La alternativa sería agregar una columna `es_unranked` por jugador —el código ya la calcula internamente, sólo no la escribe al CSV— para que el modelo pueda tratar el caso aparte. Está deliberadamente pendiente para discutir en la Unidad 2, cuando la exploración muestre qué proporción de jugadores cae en esa situación. Reconocer una decisión abierta es mejor que fingir que no existe.

6. Las capas en disco

```
include/output/
├── bronze/                                  ← CAPA BRONCE
│   ├── ddragon/version=16.17.1/
│   │   ├── champion.json.gz                 catálogo de campeones
│   │   └── summoner.json.gz                 catálogo de hechizos
│   ├── matches/plataforma=la2/
│   │   ├── LA2_1617259431.json.gz           una partida cruda
│   │   └── _manifiesto.txt                  progreso de la corrida
│   ├── players/plataforma=la2/puuid=AO23g.../
│   │   ├── liga.json.gz                     ranking del jugador
│   │   └── maestria_875.json.gz             maestría con ese campeón
│   └── descartadas/plataforma=la2.txt       ids que no volver a bajar
│
├── silver/_parciales/                       ← CAPA PLATA
│   └── plataforma_la2.csv                   filas de una plataforma
│
├── _consolidado.csv                         todas, deduplicadas
├── lol_2026-08-27.csv                       ← EL ENTREGABLE
└── reports/calidad.md                       el chequeo de la consigna
```

POR QUÉ EL BRONCE ESTÁ PARTICIONADO ASÍ

Las carpetas con nombre `clave=valor` son **particionado estilo Hive**, el mismo idioma que usan los data lakes sobre S3 pero con carpetas en vez de prefijos de bucket. Permite borrar una plataforma entera sin tocar las demás, y saber qué contiene un archivo sólo por su ruta.

POR QUÉ TODO ESTÁ COMPRIMIDO

Los JSON de Riot son texto muy repetitivo y comprimen muchísimo. Con 19.145 partidas y 290.654 perfiles, el bronce ocupa **1,1 GB**. Sin comprimir pasaría holgadamente los 8 GB. El detalle de una partida pesa unos 11 KB comprimido.

ESTADO ACTUAL DEL BRONCE

Medido el 6/9/2026, después de la recolección grande. La columna de partidas cuenta los archivos de cada carpeta `plataforma=`; unas pocas partidas quedan guardadas bajo una plataforma distinta a la de su `match_id`, porque un jugador semilla puede tener partidas jugadas en otra región.

PLATAFORMA	PARTIDAS	PERFILES
`kr` (KR)	3.430	52.455
`br1` (BR)	3.356	45.160
`la1` (LAN)	3.355	35.366
`la2` (LAS)	3.066	52.928
`na1` (NA)	2.988	53.128
`euw1` (EUW)	2.950	51.617
Total	**19.145**	**290.654**

Más 1.651 partidas descartadas —remakes, partidas de menos de cinco minutos, colas que no son la 420— anotadas en `bronze/descartadas/` para no volver a pedirlas nunca. Ese índice es el que más cuota ahorra: el detalle de partida es la respuesta más pesada de la API, y sin él cada partida basura se volvería a bajar cada vez que otro jugador semilla la menciona.

PARTE VI

Guía de ejecución

Esta parte está escrita para que cualquiera del grupo pueda levantar el proyecto en su máquina desde cero y llegar al CSV completo, sin haber participado del desarrollo.

1. Requisitos previos

HERRAMIENTA	PARA QUÉ	CÓMO INSTALARLA EN WINDOWS
Docker Desktop	Airflow corre dentro de contenedores.	Desde `docker.com`. Tiene que estar **abierto y corriendo** antes de arrancar.
Astro CLI	Levanta el entorno de Airflow.	`winget install -e --id Astronomer.Astro`
Git	Clonar el repositorio.	`winget install -e --id Git.Git`
Python 3.11+	Sólo para verificar el CSV desde afuera. Opcional.	`winget install -e --id Python.Python.3.11`

Espacio en disco: la imagen de Astro Runtime pesa ~1,5 GB y PostgreSQL ~0,45 GB. Con 6 a 8 GB libres alcanza cómodo. El bronce crece aparte: estimamos ~70 KB por partida, así que 20.000 partidas son alrededor de **1,4 GB**.

2. Puesta en marcha desde cero

```
git clone https://github.com/aguilarezequiel/FF-15.git
cd FF-15

# La configuración con la key NO viene en el repo. Se copia de la plantilla:
cp airflow_settings.example.yaml airflow_settings.yaml
# (en PowerShell:  Copy-Item airflow_settings.example.yaml airflow_settings.yaml)

# Editar airflow_settings.yaml y pegar la key en conn_password  (ver punto 3)

astro dev start
```

El primer arranque tarda unos minutos porque construye la imagen. Cuando termina imprime:

```
Airflow UI: http://localhost:8080
Added Connection: riot_api
```

Se entra a `http://localhost:8080` con usuario `admin` y contraseña `admin`. El DAG `lol_ingest` tiene que aparecer en la lista.

COMANDOS QUE SE USAN A DIARIO

COMANDO	CUÁNDO
`astro dev start`	Levantar el entorno.
`astro dev stop`	Apagarlo sin borrar nada.
`astro dev restart`	**Sólo** si se tocó `requirements.txt`, `packages.txt`, el `Dockerfile` o `airflow_settings.yaml`. Cambiar un `.py` no lo necesita.
`astro dev parse`	Verificar que los DAGs no tengan errores de sintaxis.
`astro dev pytest`	Correr los 10 tests.
`astro dev logs -f`	Ver los logs de los contenedores.

3. Obtener y renovar la API key

1. Entrar a https://developer.riotgames.com e iniciar sesión con la cuenta de Riot.
2. **Recargar la página** (importante: a veces muestra una key vieja cacheada).
3. Apretar `REGENERATE API KEY`.
4. Copiar la key que aparece — empieza con `RGAPI-` y tiene 42 caracteres.
5. Pegarla en `airflow_settings.yaml`, en el campo `conn_password`.
6. `astro dev restart` para que Airflow la recargue.

> DOS COSAS QUE NOS PASARON Y TE VAN A PASAR
>
> **La key tarda hasta un minuto en activarse.** Si la probás apenas la generás, Riot responde `401 Unknown apikey`. No está rota: no se propagó todavía. Esperá un minuto y reintentá. (El pipeline maneja esto solo: el sensor sondea cada 2 minutos.)
>
> **Regenerar invalida la anterior.** Si dos integrantes regeneran desde la misma cuenta, el segundo le rompe la key al primero. Coordinen: **cada integrante con su propia cuenta de Riot**.

EL POOL DE KEYS — CÓMO ACELERAR LA RECOLECCIÓN

Los límites de Riot son por key **y** por región. Con varias keys, el cupo se multiplica. Se pegan todas en el mismo campo, separadas por coma:

```
conn_password: RGAPI-aaaa-...,RGAPI-bbbb-...,RGAPI-cccc-...
```

El cliente arma el pool solo y reparte los pedidos. **Es la palanca más efectiva que tiene el grupo para acortar el tiempo de recolección.**

4. Cómo llegar a 20.000 filas

LOS NÚMEROS MEDIDOS *(CORRIDA REAL DEL 5–6/9/2026)*

Estos ya no son estimaciones: salen de la corrida grande, con **7 keys en el pool** (6 vivas) y las 6 plataformas en paralelo, medidos sobre 16.908 partidas y 283.522 pedidos a Riot.

MEDICIÓN	VALOR MEDIDO
Pedidos por partida	**16,8** promedio (1 de partida + ~16 de perfiles)
Ritmo agregado, 6 plataformas	**~1.154 partidas/hora**
Tiempo real para ~17.000 partidas	**14 h 39 min**
Capa plata sobre 19.145 partidas	**~13 min** (las 6 plataformas en paralelo)
DAG completo con el bronce ya en disco	**15 min** de punta a punta

El costo por partida **baja a medida que el dataset crece**, porque los perfiles de jugadores repetidos salen de la caché en vez de la API. Se ve clarísimo comparando las plataformas que llegaron al final con las que se cortaron antes:

PLATAFORMA	PARTIDAS	PEDIDOS	PEDIDOS POR PARTIDA
LAN	3.334	39.097	**11,7**
BR	3.334	48.951	14,7
EUW	2.540	47.633	18,8
LAS	2.610	49.293	18,9
NA	2.570	49.556	19,3
KR	2.520	48.992	**19,4**

> CUIDADO: AGREGAR KEYS NO ACELERA PROPORCIONALMENTE
>
> Una versión anterior de este documento proyectaba ~6.400 partidas/hora con 6 keys, extrapolando linealmente desde una. **La corrida real dio 1.154/hora con 7 keys** — cinco veces y media menos. La extrapolación lineal no se cumple y planificar con ella lleva a subestimar mucho el tiempo.
>
> La explicación más probable, leyendo el código: el techo no lo pone el cupo de las keys sino la **concurrencia del cliente**. Dentro de cada tarea las partidas se bajan una por una en un bucle secuencial, y sólo los perfiles de cada partida van en paralelo, con `ThreadPoolExecutor(max_workers=4)`. Sumar keys agranda un cupo que no se está llegando a consumir. Si en algún momento hace falta acelerar de verdad, el cambio que rinde es subir la concurrencia, no juntar más keys.
>
> **Para planificar:** contar con ~1.150 partidas/hora. 20.000 partidas son unas **17 horas** de reloj, y una development key vive 24. Entra, pero sin margen para imprevistos — de ahí la estrategia por tandas de acá abajo.

> **DESACTIVAR LA SUSPENSIÓN DE LA MÁQUINA ANTES DE UNA CORRIDA LARGA**
>
> El 6/9 la máquina hibernó a las 03:19 y mató cuatro tareas que llevaban 14 horas corriendo. El bronce sobrevivió intacto y Airflow relanzó las tareas solo, así que no se perdió nada — pero se perdieron horas de reloj contra una key que vence. Antes de largar una tanda de varias horas, desactivar suspensión e hibernación en la configuración de energía de Windows.

LA ESTRATEGIA RECOMENDADA: POR TANDAS

En vez de una corrida única de 20.000, conviene hacer varias más chicas. Una corrida única es todo-o-nada: si falla una plataforma cerca del final, `consolidate` no corre y no sale ningún CSV, aunque el bronce esté sano en disco.

Con tandas, en cambio, **cada una es una corrida en verde completa que deja un CSV válido** y suma al bronce. En todo momento hay un entregable presentable.

TANDA	`PARTIDAS_OBJETIVO`	TOTAL ACUMULADO	POR QUÉ
1	3.000	~3.000	Supera el mínimo de 1.000 de la consigna. Ya hay entregable.
2	5.000	~8.000	Zona cómoda para modelar en la Unidad 3.
3	6.000	~14.000	
4	6.000	~20.000	Objetivo alcanzado.

> **POR QUÉ ESTO FUNCIONA**
>
> El bronce es **acumulativo y resumible**. Cada tanda suma partidas nuevas al disco, y `refine_silver` reconstruye la plata sobre *todo* el bronce acumulado. Si la key vence a mitad de una tanda, se genera una nueva y la siguiente corrida sigue desde donde quedó. Nada se pierde.

CÓMO DISPARAR UNA TANDA DESDE LA INTERFAZ

1. Entrar a `http://localhost:8080`.
2. Buscar el DAG `lol_ingest` y despausarlo con el interruptor de la izquierda.
3. Apretar el botón de **Trigger** y elegir **Trigger DAG w/ config**.
4. Completar el formulario:
   - `partidas_objetivo`: el número de la tanda
   - `plataformas`: dejar las seis
   - `tiers`: dejar los diez
   - `forzar`: `false`
5. Confirmar. La corrida arranca y se puede seguir en la vista **Graph**.

Y DESDE LA TERMINAL, SI SE PREFIERE

```
astro dev run dags unpause lol_ingest
astro dev run dags trigger lol_ingest --conf '{"partidas_objetivo":3000}'
```

AL TERMINAR CADA TANDA, VERIFICAR

```
python -c "import pandas as pd, glob; \
f=sorted(glob.glob('include/output/lol_*.csv'))[-1]; \
d=pd.read_csv(f); print(f, d.shape, 'clave unica:', d['match_id'].is_unique)"
```

Y leer el reporte que dejó la corrida en `include/output/reports/calidad.md`.

5. Qué mirar mientras corre

DÓNDE	QUÉ MUESTRA
Vista **Graph**	El grafo con el color de cada tarea. Las seis instancias de `land_bronze` aparecen con el nombre de su plataforma (LAS, NA, BR…).
Logs de `land_bronze`	El progreso real. Cada 10 partidas escribe una línea con el avance y los pedidos hechos a Riot.
Carpeta `include/output/bronze/`	Los archivos aparecen a medida que llegan.

Una línea de progreso típica:

```
LAS · 120/500 partidas en bronce (118 nuevas, 2 reusadas, 6 descartadas) · 2231 requests a Riot
```

QUÉ SIGNIFICA CADA COLOR

COLOR	ESTADO	QUÉ QUIERE DECIR
Verde	success	Terminó bien.
Celeste claro	running	Está corriendo.
Rosa	skipped	Se salteó. **No siempre es un problema**: si `check_key` cortó, todo lo de abajo queda así y es el comportamiento correcto.
Rojo	failed	Falló. Hay que abrir el log.
Gris	queued	Esperando lugar para ejecutarse.

> REGLA QUE CONVIENE ADOPTAR
>
> **Leer el log, no sólo el color del cuadradito.** Un *skipped* puede ser el comportamiento correcto o un bug silencioso, y sólo el log lo distingue.

6. Problemas frecuentes y cómo resolverlos

SÍNTOMA	CAUSA	SOLUCIÓN
El DAG no aparece en la interfaz	El `dag-processor` escanea cada tanto. Si persiste, hay un error de sintaxis (aparece un banner rojo).	`astro dev parse`. La causa más común en Airflow es **olvidarse la llamada a la función al final del archivo**.
`check_key` corta y todo queda salteado	La API key venció, es inválida, o se generó hace menos de un minuto.	Leer el log de `check_key`: dice exactamente qué pasó. Regenerar la key, actualizar `airflow_settings.yaml` y `astro dev restart`.
La recolección va muy lenta	Una sola key.	Sumar keys del grupo separadas por coma.
`consolidate` falla con «ninguna plataforma produjo filas»	Ninguna tarea de descarga consiguió partidas.	Mirar los logs de `land_bronze`. Casi siempre es la key.
`validate` avisa que hay menos de 1.000 filas	Todavía no se juntó volumen suficiente.	No es un error: el DAG termina en verde igual. Correr más tandas.
Tareas clavadas en `queued`	El scheduler se cayó.	`astro dev logs -f` para ver el contenedor del scheduler; si hace falta, `astro dev restart`.
`PermissionError` al escribir en `include/`	El contenedor corre como usuario `astro`.	En Linux o WSL: `chmod -R 777 include/output`. En Windows con Docker Desktop no suele pasar.
Todo se cuelga sin mensaje	Espacio en disco.	Verificar cuánto queda libre.

CÓMO CANCELAR UNA CORRIDA EN MARCHA

Desde la interfaz: abrir la corrida y marcarla como *failed*. Desde la terminal, usando la API de Airflow:

```
TOKEN=$(curl -s -X POST http://localhost:8080/auth/token \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin"}' | python -c "import sys,json;
print(json.load(sys.stdin)['access_token'])")

curl -s -X PATCH "http://localhost:8080/api/v2/dags/lol_ingest/dagRuns/<RUN_ID>" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"state":"failed"}'
```

Cancelar no destruye nada: el bronce ya descargado queda en disco y la próxima corrida lo reutiliza.

7. Antes de la entrega — lista de control

	QUÉ
☐	El CSV tiene **más de 1.000 filas** (idealmente más de 10.000).
☐	Hay una **corrida en verde completa** en el historial, hecha por el grupo.
☐	El reporte `include/output/reports/calidad.md` está generado y leído.
☐	La API key está **vigente** (se regenera el mismo día de la entrega).
☐	Airflow arranca sin errores: probarlo la noche anterior.
☐	El CSV está abierto o listo para abrir.
☐	Todos los integrantes leyeron este documento y saben responder la parte VII.
☐	Se definió quién explica qué (parte VII.6).

> EL DÍA DE LA ENTREGA
>
> Llegar con Airflow **ya levantado** y la corrida **ya hecha**. Arrancar el entorno o correr el pipeline en vivo se come la mitad de los veinte minutos, y la consigna lo dice explícitamente entre las cosas que no hay que hacer.

PARTE VII

La defensa de la Entrega 1

Esta parte es la preparación para los veinte minutos con el docente. La evaluación es **individual**: las preguntas van dirigidas a integrantes puntuales, así que todo el grupo tiene que poder responder lo que sigue.

1. Cómo se estructuran los 20 minutos

MINUTOS 0–2 · DE QUÉ SE TRATA

Tres oraciones, sin filminas. Un guion posible:

> «Queremos saber si se puede predecir cuánto va a durar una partida de League of Legends sabiendo únicamente lo que se conoce antes de que arranque: quiénes juegan, con qué campeones y con qué hechizos. La fuente es la API oficial de Riot Games. **Una fila de nuestro dataset es una partida clasificatoria de la Season 16**, identificada por su `match_id`, y la columna que queremos predecir es su duración en segundos.»

MINUTOS 2–7 · LA CORRIDA

Se muestra la ejecución en verde en la vista *Graph* y se recorre el grafo explicando **qué hace cada tarea y por qué está donde está**. Un recorrido posible:

1. `wait_for_riot` — «Antes de nada verificamos que la API responda y que la key sirva. Es un sensor: espera algo que está fuera de nuestro control. Está primero a propósito, porque la corrida puede durar horas y enterarnos al final de que la key venció sería el peor resultado.»
2. `check_key` — «Si la key no sirve, corta acá y la corrida termina bien, no fallada. Distinguimos que Riot esté caído —donde esperar sirve— de que la key esté vencida —donde esperar no arregla nada—.»
3. `fetch_ddragon` — «Bajamos el catálogo de campeones del CDN público. Va a bronce, particionado por versión, porque los campeones cambian entre parches.»
4. `plan_plataformas` — «Repartimos el objetivo entre los seis servidores. Riot aplica los límites por región, así que paralelizar por servidor multiplica la velocidad.»
5. `land_bronze` — «Estas seis tareas son la única parte del pipeline que toca la red. Guardan el JSON tal como llegó, sin interpretarlo. Una partida que ya está no se vuelve a pedir nunca.»
6. `refine_silver` — «Acá pasa la transformación, y no toca la red: lee del bronce. Por eso podemos corregir un error de parseo y reprocesar sin gastar un solo pedido a la API.»
7. `consolidate` — «Junta las seis plataformas y deduplica por `match_id`. Es lo que garantiza que la clave primaria sea única.»
8. `validate` — «Chequeos duros. Si la clave repite o falta una columna obligatoria, el DAG falla: no publicamos un dataset roto. Además deja escrito el reporte de calidad.»
9. `save` — «Escribe el CSV fechado con la corrida.»

MINUTOS 7–12 · EL DATASET

Se abre el CSV y se lo defiende contra los criterios: cuántas filas, cuál es la clave, dónde hay nulos, cuál es la columna objetivo. Todo eso está en las secciones 2 y 3 que siguen.

2. Las seis verificaciones numéricas

Cada corrida deja estos números calculados en `include/output/reports/calidad.md`. Conviene tenerlo abierto.

CRITERIO	QUÉ TIENE QUE DAR	CÓMO SE VERIFICA	RESULTADO DEL 6/9
Clave sin duplicados	`True`	`df["match_id"].is_unique`	**True**
Volumen suficiente	más de 1.000 filas	`len(df)`	**19.145**
Ancho suficiente	5 o más columnas útiles	`df.shape`	**137**
Mezcla de tipos	numéricas y categóricas	`df.dtypes.value_counts()`	**106** enteras, **20** flotantes, **11** de texto
Nulos conocidos	saber cuáles y por qué	`df.isna().mean().sort_values(ascending=False)`	**ninguno**, en ninguna columna
Sin columnas vacías	ninguna al 100 %	`df.columns[df.isna().all()]`	**vacío**

Y la columna objetivo, que es lo primero que se pregunta después de los seis criterios: `game_duration_s` tiene media **1.748 s** (29 min 8 s), mediana **1.751 s**, desvío 424 s y rango de **301 a 3.963 s**. Distribución centrada y sin colas absurdas —el mínimo de 301 s es el piso que impone `DURACION_MINIMA_S` al descartar remakes.

> LA CLAVE PRIMARIA ES EL CRITERIO QUE MÁS PESA
>
> La consigna lo dice así: *«es el test operativo de la unidad de análisis: si la clave repite, o la unidad está mal definida o el pipeline está duplicando filas. No existe un dataset con la clave limpia y la unidad equivocada.»*
>
> Nuestra respuesta: la clave es `match_id`, viene de la fuente (no la inventamos nosotros), y `consolidate` deduplica explícitamente antes de publicar. Además `validate` hace fallar el DAG si llegara a repetirse.

3. Las cinco preguntas que se explican

La consigna las lista textualmente. Estas son las respuestas.

«Una fila es un ___»

Una partida clasificatoria de League of Legends, modalidad Solo/Duo, de la Season 16.

Coincide con la pregunta del proyecto: queremos predecir cuánto dura *una partida*, y la unidad sobre la que sacamos conclusiones es exactamente esa. Si nuestra unidad fuera el jugador, habría un desajuste con la pregunta.

¿Cuál es la columna objetivo?

`game_duration_s`: la duración de la partida en segundos.

Es continua, así que el problema es de **regresión**. Sale del campo `gameDuration` del detalle de partida, con una corrección: Riot devuelve ese campo en milisegundos en partidas anteriores al parche 11.20, así que normalizamos.

Como estructura latente alternativa, el dataset también permite **agrupamiento no supervisado** sobre las columnas de composición, para encontrar arquetipos de equipo.

¿Por qué hay nulos donde hay nulos?

No hay nulos, y esa es la parte que hay que explicar bien, porque no es que los datos vengan perfectos.

Donde la fuente no tiene el dato, imputamos un valor que significa algo: un jugador sin ranking queda con `player_elo = 1500` y `winrate = 0.50` («habilidad desconocida, asumimos promedio»); un jugador que nunca usó ese campeón queda con `champ_mastery_pts = 0`, que es literalmente cierto.

La contra, que conviene decir antes de que la pregunten: imputar con 1500 mezcla «es de nivel medio» con «no sabemos nada de él». La alternativa sería una columna `es_unranked` explícita — el código ya la calcula internamente, sólo no la escribe. Está pendiente para la Unidad 2, cuando la exploración muestre qué proporción de jugadores cae ahí.

¿Qué pasa si lo corrés de nuevo? ¿Sale el mismo archivo?

No: sale uno más grande, y es a propósito.

Hay que separar dos cosas. De `refine_silver` para abajo el pipeline es una **función pura del disco**: con el mismo bronce produce siempre exactamente el mismo CSV. Pero `land_bronze` es **acumulativo**: cada corrida busca partidas que todavía no tiene. Las que ya están nunca se vuelven a pedir; las nuevas se suman.

Es la única forma de juntar volumen con una API que permite 100 pedidos cada 2 minutos por región y una key que vence cada 24 horas. Si quisiéramos congelar el dataset, alcanza con no volver a correr la descarga y reprocesar el bronce tal como está.

¿Dónde guardás el dato crudo? ¿Tenés el bronce?

Sí, y guardamos más que el mínimo. En `include/output/bronze/` queda la respuesta JSON de la API tal como llegó, comprimida y particionada por plataforma.

No guardamos sólo las partidas: guardamos **las tres respuestas** que hacen falta para reconstruir una fila —el detalle de la partida, la entrada de ranking de cada jugador y su maestría— más el catálogo de campeones. Por eso la capa plata se reconstruye **100 % sin red**.

La prueba concreta: si mañana descubrimos que veníamos interpretando mal una columna, corregimos `transform.py` y reprocesamos los **1,1 GB** que ya están en disco, **sin gastar un solo pedido a Riot**. Son ~13 minutos de reproceso contra las **14 h 39 min** que costó bajarlos.

4. La confesión sobre tidy

La consigna pide explícitamente declarar dónde el dataset se aparta del ideal: *«Si tu dataset tiene una concesión parecida, no la escondas: explicala. Un grupo que sabe dónde su dataset se aparta del ideal y por qué está mejor parado que uno que repite la definición de memoria.»*

> NUESTRA CONCESIÓN
>
> El dataset **no es del todo tidy**. La observación más fina que existe acá es *jugador-en-partida*: hay diez por partida. Están **aplanados en 100 columnas** (`t1_top_champ_id` … `t2_sup_is_main`) en lugar de ocupar diez filas.

POR QUÉ ES UNA CONCESIÓN DELIBERADA Y NO UN DESCUIDO

1. **La unidad de la pregunta es la partida, no el jugador.** Queremos predecir cuánto dura *una partida*. En formato largo habría diez filas por partida con el target `game_duration_s` repetido idéntico en las diez — lo cual rompe la correspondencia una-fila-una-observación justo respecto de la pregunta que importa.
2. **Obligaría a re-agregar antes de cada modelo.** Todo modelo de regresión sobre la duración necesitaría volver a juntar las diez filas en una. La agregación quedaría duplicada en cada análisis en vez de estar resuelta una sola vez en el pipeline.
3. **Es el mismo tipo de concesión que hace la cátedra.** El dataset FIFA guarda varios valores en la celda `player_positions` en el 65 % de las filas, por la misma razón: normalizarlo complicaría todo lo demás.

QUÉ SÍ CUMPLE

Los otros dos criterios eliminatorios relacionados están sólidos: la **unidad de análisis está alineada** con la pregunta (una fila = una partida = aquello sobre lo que concluimos), y **cada columna es una variable** — ninguna celda guarda listas ni valores múltiples, que es el problema que tiene el dataset de la cátedra.

5. Limitaciones que declaramos

Decirlas antes de que las pregunten es mejor que que las descubran.

1 · FUGA TEMPORAL EN WINRATE Y MAESTRÍA

`player_winrate` y `champ_mastery_pts` se consultan **en el momento de la recolección**, no en el momento en que se jugó la partida. Para una partida de hace semanas son valores *posteriores* al hecho, lo cual tensiona el claim de «sólo variables pre-partida».

Por qué no se puede arreglar: la API de Riot no expone winrate ni maestría históricos. **Qué se puede hacer:** acotar el efecto recolectando partidas recientes, que es lo que hacemos al pedir las últimas 15 de cada jugador. Y declararlo, que es lo que estamos haciendo acá.

2 · `PLAYER_ELO` ES UNA CONSTRUCCIÓN NUESTRA

Riot no publica el MMR. Nuestra fórmula convierte tier, división y puntos en un número comparable, pero los coeficientes son una elección del grupo, no una medición. Hay que presentarla como variable derivada, no como dato observado. Si en la Unidad 3 no aporta, se reemplaza — el bronce permite rehacerlo sin costo de API.

3 · EL SESGO DE MUESTREO POR TIER *(MEDIDO EL 6/9/2026)*

Es la limitación más importante del dataset y la que hay que saber explicar. **Cada plataforma quedó con partidas de un rango casi puro, y cada una de un rango distinto.**

PLATAFORMA	RANGOS DOMINANTES	ELO MEDIO	DESVÍO
`la1` (LAN)	GRANDMASTER 39% · MASTER 23% · DIAMOND 21%	3.231	293
`br1` (BR)	DIAMOND 49% · GRANDMASTER 22% · MASTER 14%	3.036	262
`euw1` (EUW)	EMERALD 48% · DIAMOND 33%	2.585	223
`kr` (KR)	SILVER 29% · BRONZE 26% · **CHALLENGER 25%**	2.254	**1.293**
`la2` (LAS)	GOLD 37% · PLATINUM 32%	2.178	440
`na1` (NA)	SILVER 33% · BRONZE 29%	1.627	342

Por qué pasa. `iter_puuids` mezcla el orden de los tiers justamente para evitar un sesgo, pero el shuffle no alcanza: una página de escalera trae **205 jugadores** y cada uno rinde hasta 15 partidas, así que **una sola combinación tier/división da unas 3.000 partidas** — la cuota entera de la corrida. La corrida corta antes de llegar al segundo tier y el orden mezclado nunca se recorre. Los logs lo muestran directo: la corrida de KR del 5/9 entró por `BRONZE I` y `BRONZE II` y no tocó nada más; al reintentarse tras la hibernación entró por `CHALLENGER I`. Por eso KR es la única plataforma bimodal, y por eso su desvío (1.293) triplica al de las demás.

Qué implica. Dos cosas concretas, y conviene decirlas antes de que las pregunten:

- **El poder predictivo del elo está subestimado.** Dentro de cada plataforma casi no varía, así que el modelo tiene poco de dónde aprender. La correlación elo↔duración es `-0,16` global, pero `-0,25` en KR —la única con rango amplio— y apenas `-0,01` en NA1, donde todos los jugadores tienen prácticamente el mismo rango. La señal aparece donde hay variación y desaparece donde no la hay: es la firma del sesgo, no una propiedad del juego.
- **Plataforma y elo están confundidos.** En NA las partidas duran en promedio 1.826 s y en BR 1.687 s, pero eso *no* es un efecto de la región: es que NA quedó lleno de plata/bronce y BR de diamante. Atribuirlo al servidor sería una conclusión falsa, y es justo el error que la materia señala como «el modelo aprende el sesgo de recolección en vez de la señal».

Cómo se arregla (fuera del alcance de la Entrega 1): rotar de tier cada N jugadores dentro de la corrida, en vez de confiar en que el shuffle del orden alcance. El bronce ya bajado se conserva, así que sumar tiers faltantes no obliga a rehacer nada.

4 · LA SEMILLA NO ES LA MUESTRA

Aparte de lo anterior: los jugadores se muestrean recorriendo el ranking, pero de cada partida entran los diez participantes sin importar su rango. La distribución de rangos del dataset **no es** la de la semilla. Como el emparejamiento junta gente de nivel parecido no distorsiona mucho —de hecho es lo que hace que el sesgo del punto 3 se propague de la semilla a la partida entera—, pero es una diferencia real entre cómo elegimos y qué terminamos midiendo.

5 · EL UMBRAL DE `IS_MAIN`

50.000 puntos de maestría es un corte elegido por el grupo, sin análisis de sensibilidad. `champ_mastery_pts` está igual en el dataset, así que en la Unidad 3 se puede dejar que el modelo encuentre el corte en vez de imponerlo.

6. Reparto de conocimiento en el grupo

La consigna es tajante: *«Que hable uno solo es el error más caro, porque las preguntas van dirigidas y la nota es individual.»*

Propuesta de reparto. **Completar con los nombres del grupo** y que cada uno lea al menos las partes indicadas. Todos deberían poder responder las cinco preguntas de la sección 3 — eso no se reparte.

ÁREA	QUIÉN	QUÉ TIENE QUE DOMINAR	PARTES A LEER
La fuente	_____	Qué es la API de Riot, los límites y el vencimiento de la key, las dos familias de hosts, los siete endpoints, y el recorrido de semilla a dataset.	III completa
La extracción	_____	`riot.py` y `seeds.py`: el pool de keys, el limitador proactivo, el manejo de errores, y por qué los tiers se recorren en orden aleatorio.	III, IV.4.1, IV.4.3
La transformación	_____	`transform.py` y `schema.py`: los filtros de admisión, la corrección de milisegundos, la asignación de carriles y `roles_inferidos`, y cómo se generan las 137 columnas.	IV.4.4, IV.4.5, V
La orquestación	_____	El grafo completo, qué hace cada una de las nueve tareas y por qué está donde está. Sensores, cortocircuito, trigger rules, mapeo dinámico.	II, IV.5
El dataset	_____	Los seis criterios medibles con sus números, los nulos y su justificación, la confesión de tidy.	V, VII.2, VII.3, VII.4
La operación	_____	Levantar el entorno, renovar la key, disparar una tanda, leer los logs, resolver los problemas frecuentes. **Es quien lleva la notebook el día de la entrega.**	VI completa

CÓMO PREPARARSE EN GRUPO

Juntarse una vez antes del 9/9 y hacer un simulacro: cada uno explica su área en dos minutos y los demás preguntan. Las preguntas de la sección 7 sirven de guion. Lo que no se puede explicar en voz alta, no se entendió.

7. Banco de preguntas probables

¿Por qué eligieron Airflow y no un script de Python?

Porque la corrida dura horas y necesita sobrevivir a interrupciones. Airflow nos da reintentos por tarea, logs separados, paralelismo entre las seis plataformas, estado visible y la posibilidad de retomar sin repetir lo hecho. Con un script monolítico habría que empezar de cero.

Y lo probamos sin querer. El 6/9 a las 03:19 la máquina entró en hibernación con cuatro tareas `land_bronze` a mitad de camino (entre 2.520 y 2.610 partidas de 3.334 cada una). Los procesos murieron en seco —los logs se cortan sin traceback—. **No se perdió un solo archivo del bronce.** A las 07:16 el scheduler detectó las tareas huérfanas y las relanzó solo, sin que nadie tocara nada. El dataset se generó igual ese mismo día. Es la mejor defensa de la arquitectura: el estado vive en disco, no en memoria ni en XCom.

¿Qué pasa si corren el DAG de nuevo? ¿Vuelve a bajar todo?

No. `land_bronze` chequea `destino.exists()` antes de cada pedido: una partida que ya está en disco **no se vuelve a pedir jamás**, ni en esa corrida ni en las que vengan. Y `refine_silver` reprocesa *todo* el bronce con un `glob`, no sólo lo que bajó esa corrida, así que el dataset se reconstruye entero sobre el acumulado.

La precisión que hay que hacer, porque es fácil decirlo mal: una corrida nueva *no completa* la anterior, la *amplía*. Como cada corrida entra a la escalera por un tier distinto (ver limitación 3 en 7.5), en la práctica casi no reencuentra las partidas viejas: sale a buscar jugadores nuevos y baja partidas nuevas. En el reintento del 6/9 el contador marcó `0 reusadas` con 3.000 partidas de esa misma plataforma ya en disco. No es un fallo del chequeo —es que las partidas viejas nunca vuelven a mencionarse—, pero significa que la cuota de una corrida se mide sobre lo que *esa* corrida junta, no sobre el total acumulado.

¿La muestra es representativa? ¿Cómo eligieron las partidas?

No es representativa, y lo sabemos medido. Riot no publica un listado de partidas: publica un ranking de jugadores. Así que el camino es escalera → jugador semilla → sus últimas 15 partidas. El problema es que una página de escalera trae 205 jugadores y eso alcanza para llenar la cuota entera de una corrida, así que cada plataforma quedó con partidas de un rango casi puro: NA1 con 33% SILVER y 29% BRONZE, LA1 con 39% GRANDMASTER. El elo medio va de 1.627 a 3.231 según el servidor.

Está desarrollado con la tabla completa, la causa y las dos consecuencias en la limitación 3 de la sección 5. La respuesta corta para la defensa: *«el dataset es una muestra de conveniencia sesgada por tier; lo medimos, lo declaramos, y sabemos cómo se corrige —rotando de tier dentro de la corrida en vez de confiar en el shuffle del orden»*.

¿Por qué el DAG no está programado?

Porque la development key de Riot vence cada 24 horas. Un DAG programado corriendo sin nadie mirando fallaría todos los días apenas venciera la key, y el ruido de esas fallas taparía las reales. Se dispara a mano con la key recién renovada. El DAG de ejemplo de la cátedra, `pokemon_api`, también usa `schedule=None`.

¿Por qué seis tareas de descarga en vez de una?

Porque Riot aplica los límites **por región**: cada plataforma tiene su propio cupo. Seis tareas en paralelo multiplican por seis la velocidad en los endpoints de perfil y maestría, que son el cuello de botella real: medimos **16,8 pedidos por partida** sobre 16.908 partidas, de los cuales ~16 son perfiles.

¿Dónde está la API key?

En una Connection de Airflow llamada `riot_api`, declarada en `airflow_settings.yaml`, que está en `.gitignore`. En el repositorio se versiona `airflow_settings.example.yaml` con un placeholder. **La key no aparece en ningún `.py`.** El campo acepta varias keys separadas por coma, una por integrante, y el cliente arma un pool.

¿Qué pasa si Riot está caído cuando corren el pipeline?

El sensor `wait_for_riot` sondea cada 2 minutos durante media hora antes de rendirse. Si se agota, queda *salteado* —no fallado—, `check_key` corta y la corrida termina bien sin hacer nada. Distinguimos ese caso del de una key inválida: ahí no esperamos, cortamos en el acto, porque esperar no arregla una key vencida.

¿Por qué descartan las partidas de menos de 5 minutos?

Son *remakes*: partidas que se cancelan al principio porque alguien no se conectó. No son partidas reales y su duración no tiene nada que ver con las variables que estamos midiendo. Incluirlas metería ruido en el objetivo. Además, una vez descartada, la partida queda anotada en un índice y no se vuelve a descargar nunca: era el mayor desperdicio de cupo del prototipo anterior.

¿Cómo saben que no hay fuga de información?

Porque el esquema excluye por diseño todo lo que ocurre durante la partida: no hay kills, ni oro, ni torres, ni deltas por minuto, ni bans. Sólo entran datos del draft (campeones, hechizos, runas) y perfiles de jugador.

Dicho eso, hay una fuga acotada que declaramos: winrate y maestría se consultan hoy, no al momento de la partida. La API no ofrece histórico, así que no se puede resolver con esta fuente.

Si les cambio el parseo de una columna, ¿tienen que volver a descargar todo?

No, y ésa es exactamente la razón de tener capa bronce. Se corrige `transform.py`, se vuelve a disparar el DAG y `refine_silver` reconstruye la plata leyendo del disco, sin tocar la red. Con 20.000 partidas eso son horas de descarga ahorradas.

¿Por qué el CSV tiene 137 columnas y no las 138 del prototipo?

Dos cambios. **Sumamos** `roles_inferidos`: el prototipo asignaba los carriles buscando `teamPosition`, y cuando venía vacío —remakes, autofill— mandaba al jugador al slot que sobrara **sin dejar rastro**. Ahora contamos cuántos quedaron así y esas filas se pueden filtrar. **Quitamos** `t1_count_smite` y `t2_count_smite`: el juego obliga al jungla a llevar Smite, así que valían 1 en toda partida real. Una columna de varianza cero no aporta a un modelo. Neto: 138 + 1 − 2 = 137.

¿Qué es esa columna `t1_num_tanks` y por qué las seis no suman 5?

Cuenta cuántos de los cinco campeones del equipo están etiquetados como Tank en Data Dragon. No suman 5 porque un campeón puede tener varias etiquetas a la vez: Ornn es Tank y Fighter. No es un error de cálculo, es cómo clasifica el catálogo oficial.

¿Cómo garantizan que la clave no se repita?

Tres capas. El `match_id` viene de la fuente y ya incluye el prefijo de la plataforma, así que es único globalmente. `consolidate` hace un `drop_duplicates("match_id")` explícito al juntar las seis plataformas. Y `validate` hace fallar el DAG si aun así se repitiera.

¿Por qué no usaron un operador prefabricado para los pedidos HTTP?

Porque nuestro cliente necesita rotar entre varias API keys y llevar la cuenta de los pedidos por par (key, host) para respetar los límites de Riot. Ningún operador prefabricado hace eso. La cátedra da la regla: operador cuando la acción es estándar, `@task` propio cuando se necesita lógica específica. Sí usamos la infraestructura de Connections de Airflow para la credencial, que es la parte que sí conviene reusar.

ANEXO

Glosario

Términos que aparecen en este documento, en el código y en las clases.

De la materia y de ingeniería de datos

TÉRMINO	QUÉ ES
Pipeline	Secuencia de pasos por la que pasa el dato desde el origen hasta el destino.
ETL	*Extract, Transform, Load*. Se transforma antes de guardar. Se guarda sólo el resultado.
ELT	*Extract, Load, Transform*. Se guarda el dato crudo primero y se transforma después. **Es lo que hacemos.**
Modelo medallón	Organización en capas: bronce (crudo), plata (limpio) y oro (listo para un uso específico).
Bronce	El dato tal como llegó de la fuente, sin interpretar. Sólo se agrega, nunca se modifica.
Plata	Datos limpios, tipados y validados, una fila por observación. **Es nuestro entregable.**
Oro	Datos preparados para un uso concreto. Se construye en las Unidades 3 y 4.
Idempotencia	Que ejecutar algo muchas veces dé el mismo resultado que ejecutarlo una vez.
Tidy	Formato donde cada fila es una observación y cada columna una variable.
Unidad de análisis	Qué representa cada fila. En nuestro caso, una partida.
Clave primaria	La columna que identifica unívocamente cada fila. Acá, `match_id`.
Variable objetivo	La que se quiere predecir. Acá, `game_duration_s`.
Regresión	Predecir un número continuo. (Predecir una categoría es *clasificación*.)
Fuga de información	*Data leakage*. Que entre al modelo información que en la realidad no estaría disponible al momento de predecir.
Sesgo de recolección	Que la forma de juntar los datos introduzca un patrón que no existe en la realidad.
Feature engineering	Derivar variables nuevas a partir de las existentes. `player_elo` es un ejemplo.

De Airflow

TÉRMINO	QUÉ ES
DAG	*Directed Acyclic Graph*. El dibujo del flujo de trabajo: tareas unidas por dependencias, sin ciclos.
Tarea (*task*)	Una cajita del grafo. Unidad de ejecución con su log, su estado y sus reintentos.
DagRun	Una ejecución concreta del DAG. Cada disparo crea una.
Operador	Plantilla de tarea prefabricada.
Hook	La capa de abajo de un operador: lo que sabe hablar con el sistema externo.
Provider	Paquete que agrega integraciones. Se declara en `requirements.txt`.
TaskFlow API	La forma moderna de escribir DAGs, con decoradores `@dag` y `@task`.
XCom	Mecanismo para pasar datos entre tareas. **Sólo metadatos**: rutas, ids, conteos.
Connection	Donde Airflow guarda credenciales y datos de conexión. La nuestra se llama `riot_api`.
Sensor	Tarea que espera a que ocurra algo fuera del control del DAG.
`mode="reschedule"`	Modo de sensor que libera el trabajador entre sondeos en vez de ocuparlo durmiendo.
`soft_fail`	Que un sensor que agota su tiempo quede *salteado* y no *fallado*.
Short-circuit	Tarea que, si devuelve `False`, saltea todo lo que está aguas abajo y termina bien.
Trigger rule	Condición para que una tarea corra. Por defecto `ALL_SUCCESS`; nosotros usamos `ALL_DONE` en `check_key`.
Mapeo dinámico	`.expand()`: crear una instancia de la tarea por cada elemento de una lista. Lo usamos por plataforma.
`catchup`	Si Airflow debe recuperar las corridas que se perdió mientras estuvo apagado. El nuestro está en `False`.
`dag-processor`	El proceso que lee los archivos de `dags/`. En Airflow 2 vivía dentro del scheduler.
`api-server`	La interfaz web y la API REST. En Airflow 2 se llamaba `webserver`.
Astro Runtime	Imagen de Docker con Airflow ya armado. Usamos la 3.3-4.
Astro CLI	El comando que levanta esa imagen. `astro dev start` es de Astro, no de Airflow.

De League of Legends y de la API de Riot

TÉRMINO	QUÉ ES
Campeón	El personaje que controla un jugador. Hay más de 160.
Carril (*lane*)	La posición que ocupa un jugador: top, jungla, mid, bot o support. Cinco por equipo.
Draft	La fase previa donde cada equipo elige sus campeones. **Todo lo que usamos sale de acá o de antes.**
Hechizo de invocador	Habilidad extra que cada jugador elige antes de empezar. Flash (4), Smite (11), Teleport (12), Ignite (14).
Runas	Configuración de bonificaciones elegida antes de la partida. Guardamos las dos ramas.
Maestría	Puntos acumulados por un jugador con un campeón. Indica cuánto lo jugó.
Tier / división	El rango: Hierro a Challenger, con divisiones IV a I.
LP	*League Points*. Puntos dentro de una división.
MMR	*Matchmaking Rating*. El número interno con el que Riot empareja. **Riot no lo publica**; por eso construimos `player_elo`.
`puuid`	Identificador único y permanente de una cuenta.
Cola 420	El código de la modalidad Ranked Solo/Duo. La única que recolectamos.
Remake	Partida cancelada en los primeros minutos porque alguien no se conectó. Las descartamos.
Plataforma	El servidor de juego: `la2`, `na1`, `kr`... Determina el host de algunos endpoints.
Región continental	`americas`, `europe`, `asia`. Es el host de los endpoints de partidas.
Data Dragon	El CDN público de Riot con los metadatos del juego. No pide API key.
Development key	API key de prueba: 20 pedidos por segundo, 100 cada 2 minutos, **vence cada 24 horas**.
`429`	Código HTTP de «demasiados pedidos». El cliente castiga esa key en ese host y sigue con otra.

CIERRE

Qué falta y hacia dónde sigue

Lo que falta para cerrar la Entrega 1

	TAREA	QUIÉN
~~1~~	~~Que al menos dos integrantes generen su API key de Riot.~~ **Hecho:** hay 7 keys en el pool.	El grupo
~~2~~	~~Correr las tandas de recolección.~~ **Hecho el 6/9:** 19.145 partidas en bronce.	Quien opera
~~3~~	~~Verificar el reporte de calidad y que el CSV supere las 1.000 filas.~~ **Hecho:** 19.145 filas × 137 columnas, clave única, cero nulos.	Quien opera
4	Que todo el grupo lea este documento y repase la parte VII. **Prestar atención a la limitación 3 (sesgo por tier)**: es lo más probable que pregunten y lo que peor queda si nadie sabe responderlo.	Todos
5	Completar los nombres en la tabla de reparto (VII.6) y hacer un simulacro.	El grupo
6	Regenerar la key el mismo día de la entrega y verificar que Airflow levanta. **No hace falta volver a recolectar**: el bronce está en disco y una corrida con `partidas_objetivo` bajo regenera el CSV entero en ~15 minutos.	Quien lleva la notebook

> EL ENTREGABLE YA EXISTE
>
> El dataset está en `include/output/lol_2026-09-06.csv` y el reporte en `include/output/reports/calidad.md`. Lo que queda es de preparación de la defensa, no de pipeline. Si el día de la entrega hay que mostrar una corrida en vivo, conviene disparar una con `partidas_objetivo` chico: termina en minutos y produce el mismo CSV completo, porque `refine_silver` reprocesa todo el bronce acumulado.

Hacia dónde sigue el proyecto

Lo que se construyó acá es el insumo de todo lo que viene. Las decisiones que quedaron abiertas tienen su lugar natural en las entregas siguientes:

UNIDAD	QUÉ SE HACE	QUÉ DE ACÁ SE RETOMA
2 · Exploración 16/09	Explorar el dataset, formular hipótesis y encontrar hallazgos.	Evaluar si conviene una columna `es_unranked` explícita. Ver qué proporción de filas tiene `roles_inferidos > 0` (son 17 de 19.145). Analizar la distribución de la duración y buscar qué variables se le correlacionan. **Y cuantificar el sesgo por tier de la limitación 3**: es el primer hallazgo exploratorio que ya tenemos medido y el que más condiciona todo lo que venga después.
3 · Modelado 14/10	Comparar modelos de regresión sobre `game_duration_s` y medir su desempeño.	Acá aparece la **capa oro**: variables derivadas y agregados construidos sobre esta plata. Revisar si `player_elo` aporta o conviene el tier como categórica, y si `is_main` conviene o el modelo encuentra su propio corte sobre `champ_mastery_pts`.
4 · Visualización 04/11	Visualizaciones y una aplicación funcionando.	El mismo CSV alimenta los agregados de la app.

LO QUE EL BRONCE NOS DEJA DISPONIBLE

Como guardamos las respuestas crudas de la API, cualquiera de esos cambios de criterio —una columna nueva, otra fórmula de elo, otro umbral— se resuelve corrigiendo `transform.py` y reprocesando el disco, **sin gastar un solo pedido a Riot ni esperar horas de descarga**. Ésa es la razón concreta por la que el modelo medallón vale la pena, y es lo que hay que poder explicar el 9 de septiembre.
